

vLLM 内部：高通量 LLM 推理系统的剖析

从分页注意力、连续批处理、前缀缓存、specdec 等到大规模多 GPU、多节点动态服务

8月 29, 2025

在这篇文章中，我将逐步介绍构成现代高通量 LLM 推理系统的所有核心系统组件和高级功能。特别是，我将对 vLLM [1] 的工作原理进行细分。

这篇文章是系列文章中的第一篇。它从宽泛开始，然后详细分层（遵循逆金字塔方法），因此您可以形成整个系统的准确高级心智模型，而不会淹没在细节中。

后面的帖子将深入探讨特定的子系统。

这篇文章分为五个部分：

1. **LLM 引擎和引擎核心**：vLLM 基础知识（调度、分页注意力、连续批处理等）
2. **高级功能**：分块预填充、前缀缓存、引导和推测解码、分解 P/D
3. **纵向扩展**：从单 GPU 执行到多 GPU 执行
4. **服务层**：分布式/并发 Web 脚手架
5. **基准测试和自动调整**：测量延迟和吞吐量



笔记

- 分析基于[提交 42172ad](#) (2025 年 8 月 9 日)。
- 目标受众：任何对最先进的 LLM 引擎如何工作感到好奇的人，以及有兴趣为 vLLM、SGLang 等做出贡献的人。
- 我将重点介绍 [V1 发动机](#)。我还探索了 [V0 \(现已弃用\)](#)，这对于理解项目是如何演变的很有价值的，并且许多概念仍然沿袭。
- 关于 LLM 引擎/引擎核心的第一部分可能有点压倒性/枯燥 - 但博客的其余部分有很多示例和视觉效果。:)

LLM 引擎是 vLLM 的基本构建块。它本身已经实现了高通量推理，但仅限于离线设置。您还不能通过网络将其提供给客户。

我们将使用以下离线推理片段作为运行示例（改编自 [basic.py](#)）。

```
from vllm import LLM, SamplingParams

prompts = [
    "Hello, my name is",
    "The president of the United States is",
]

sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

def main():
    llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0")

    outputs = llm.generate(prompts, sampling_params)

if __name__ == "__main__":
    main()
```



环境变量：

- VLLM_USE_V1="1" # 我们使用的是引擎 V1
- VLLM_ENABLE_V1_MULTIPROCESSING="0" # 我们在单个进程中运行

此配置为：

- 离线（无 Web/分布式系统脚手架）
- 同步（所有执行都发生在单个阻塞过程中）
- 单 GPU（无数据/模型/流水线/专家并行性；DP/TP/PP/EP = 1）
- 使用标准转换器 [2]（支持 Jamba 等混合模型需要更复杂的混合 KV 缓存内存分配器）

从这里开始，我们将逐步构建一个在线、异步、多 GPU、多节点推理系统，但仍服务于标准转换器。

在这个例子中，我们做两件事，我们：

1. 实例化引擎
2. 调用它从给定的提示中采样 `generate`

让我们开始分析构造函数。

LLM 引擎构造函数

发动机的主要部件有：

- vLLM 配置（包含用于配置模型、缓存、并行度等的所有旋钮）
- 处理器（通过验证、标记化和处理将原始输入→） `EngineCoreRequests`
- 引擎核心客户端（在我们运行的示例中，我们使用的基本是 `==` ；我们将逐步构建到允许大规模服务的客户端） `InprocClient` `EngineCore` `DPLBAsyncMPCClient`
- 输出处理器（转换用户看到的原始→） `EngineCoreOutputs` `RequestOutput`



注意：

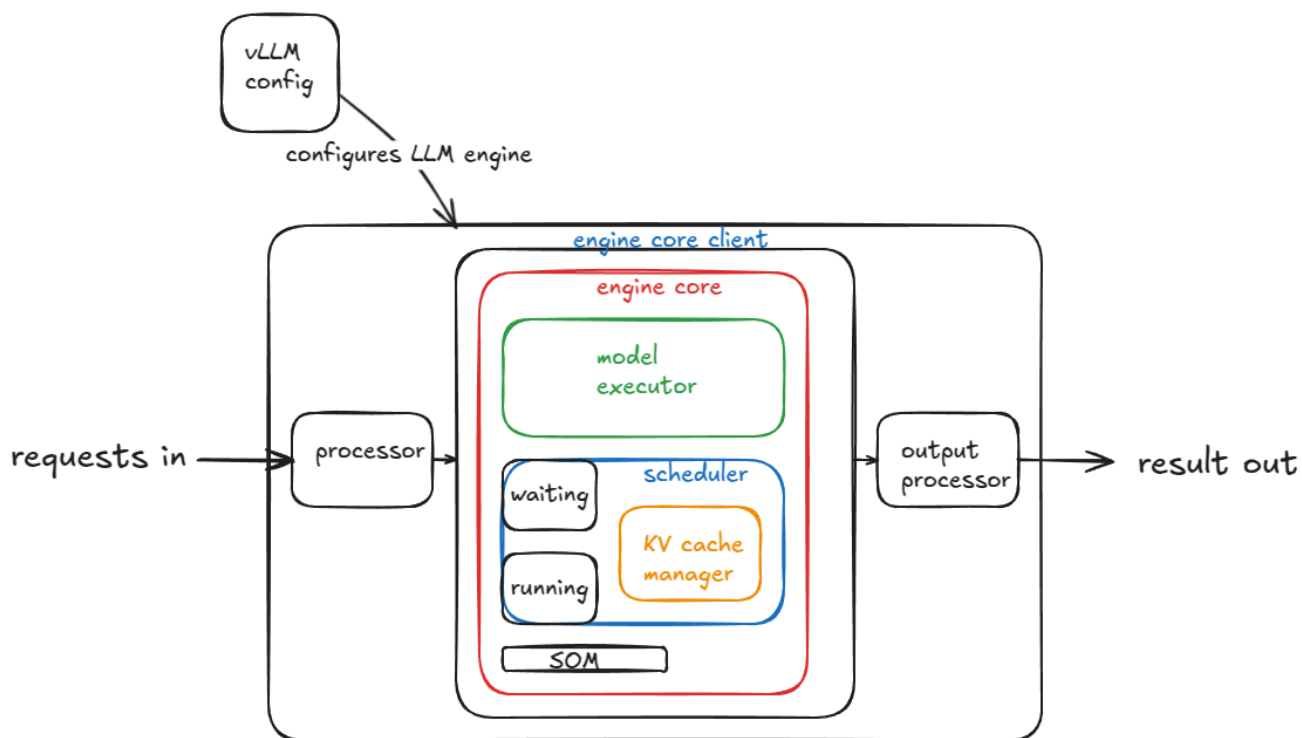
随着 V0 引擎被弃用，类名和详细信息可能会发生变化。我将强调核心思想而不是确切的签名。我将抽象出一些但不是全部这些细节。

引擎核心本身由几个子组件组成：

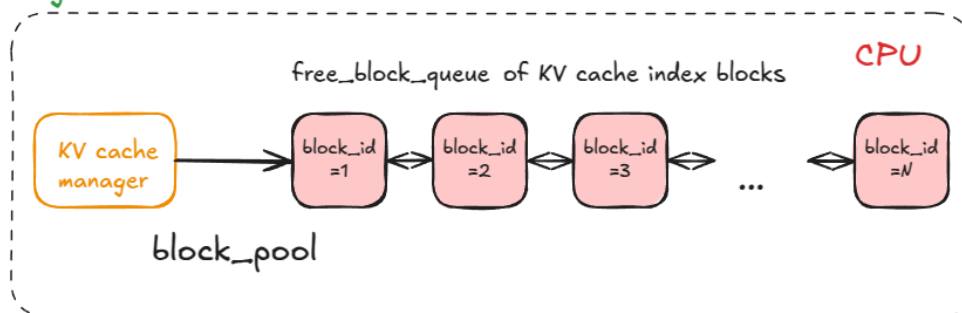
- 模型执行器（驱动模型上的前向传递，我们目前正在处理的在单个 GPU 上具有单个进程）。我们将逐步构建支持多个 GPU 的 `UniProcExecutor` `Worker` `MultiProcExecutor`
- 结构化输出管理器（用于引导解码 - 我们稍后会介绍）
- 调度程序（决定哪些请求进入下一个引擎步骤）--它进一步包含：
 - a. 策略设置 - 它可以是 **FCFS**（先到先得）或**优先级**（优先级更高的请求优先提供）
 - b. `waiting` 和队列 `running`
 - c. KV 缓存管理器 - 分页注意力的核心 [3]

KV 缓存管理器维护一个可用 KV 缓存块池（通常为数十万个，具体取决于 VRAM 大小和块大小）。在分页关注期间，这些块充当索引结构，将令牌映射到其计算的 KV 缓存块。

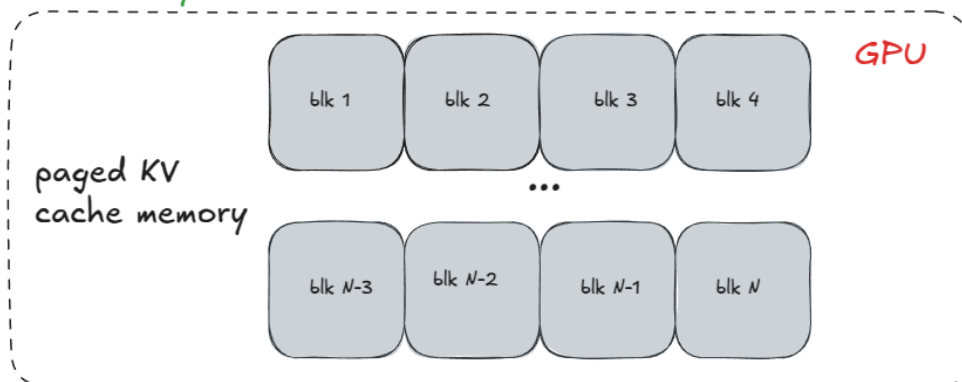
`free_block_queue`



indexing structure



actual KV memory



本节中描述的核心组件及其关系

标准 Transformer 层（非 MLA [4]）的块大小计算如下：

$2 \text{ (键/值)} * (\text{默认值}=16) * * * (\text{例如, bf16 为 } 2)$

`block_size num_kv_heads head_size dtype_num_bytes`

在模型执行器构建过程中，创建一个对象，并执行三个关键过程。（稍后，使用 ，这些相同的过程在不同 GPU 上的每个工作进程上独立运行。 `Worker MultiProcExecutor`

1. 初始化设备：

- 将 CUDA 设备（例如“cuda: 0”）分配给 worker，并检查是否支持模型 dtype（例如 bf16）
- 根据请求（例如，总 VRAM 的 0.8 → 80%），验证是否有足够的 VRAM 可用 `gpu_memory_utilization`
- 设置分布式设置（DP / TP / PP / EP等）
- 实例化一个（保存采样器、KV 缓存和前向传递缓冲区，例如 、 等）
`model_runner input_ids positions`
- 实例化对象（保存 CPU 端前向传递缓冲区、用于 KV 缓存索引的块表、采样元数据等） `InputBatch`

2. 负载模型：

- 实例化模型体系结构
- 加载模型权重
- 调用 `model.eval()`（PyTorch 的推理模式）
- 可选：在模型上调用 `torch.compile()`

3. 初始化 KV 缓存

- 获取每层 KV 缓存规范。从历史上看，这始终是（同质变压器），但随着混合模型（滑动窗口、Transformer/SSM，如 Jamba）的出现，它变得更加复杂（参见 Jenga [5] `FullAttentionSpec`）
- 运行虚拟/分析前向传递并拍摄 GPU 内存快照，以计算可用 VRAM 中适合多少 KV 缓存块
- 将 KV 缓存张量分配、重塑和绑定到注意力层
- 准备注意力元数据（例如，将后端设置为 FlashAttention），稍后在 fwd 传递期间内核会消耗这些元数据
- 除非提供，否则对于每个预热批次大小，执行虚拟运行并捕获 CUDA 图。CUDA 图将 GPU 工作的整个序列记录到 DAG 中。稍后在 fwd 传递期间，我们启动/重放预烘焙的图形并减少内核启动开销，从而改善延迟。 `--enforce-eager`

我在这里抽象了许多低级细节——但这些都是我现在要介绍的核心部分，因为我将在以下部分中反复引用它们。

现在我们已经初始化了引擎，让我们继续使用该函数。 `generate`

生成函数

第一步是验证请求并将其馈送到引擎中。对于每个提示，我们：

1. 创建唯一的请求 ID 并捕获其到达时间
2. 调用一个输入预处理器，该预处理器对提示进行标记化，并返回包含 `prompt`、`prompt_token_ids` 和 `a`（文本、标记、嵌入等）的字典。
3. 将此信息打包到一个 `EngineCoreRequest`、添加优先级、采样参数和其他元数据中
4. 将请求传递到引擎核心，引擎核心将其包装在对象中并将其状态设置为 `Request WAITING waiting`。然后将此请求添加到调度器的队列中（如果 FCFS 则追加，如果优先级则为堆推送）

此时，引擎已被馈送，可以开始执行。在同步引擎示例中，这些初始提示是我们唯一要处理的提示——没有`在运行中注入新请求的机制`。相比之下，异步引擎支持这一点（又名`连续批处理 [6]`）：在每个步骤之后，都会考虑新请求和旧请求。

由于前向传递将批处理扁平化为单个序列，并且自定义内核可以有效地处理它，因此即使在同步引擎中也从根本上支持连续批处理。

接下来，只要有要处理的请求，引擎就会重复调用其函数。每个步骤都有三个阶段：`step()`

1. 计划：选择在此步骤中要运行的请求（解码和/或（分块）预填充）
2. 前向传递：运行模型和示例令牌
3. 后处理：将采样的令牌 ID 附加到每个、取消标记化和检查停止条件。如果请求完成，请清理（例如将其 KV 缓存块返回到 `Request free_block_queue`）并提前返回输出



停止条件为：

- 请求超出其长度限制（或它自己的 `max_model_length max_tokens`）
- 采样令牌是EOS ID（除非启用 `ignore_eos` ->当我们想要强制生成一定数量的输出令牌时，这对于基准测试很有用）
- 采样的令牌与采样参数中指定的任何标记匹配 `stop_token_ids`
- 停止字符串存在于输出中 - 我们截断输出，直到出现第一个停止字符串，并在引擎中中止请求（请注意，输出中会出现，但停止字符串不会）。 `stop_token_ids`

"Hello, my name is"



```
{  
  "prompt": "Hello, my name is",  
  "prompt_token_ids": [1,2,3,4,5],  
  "type": "token"  
}  
+ misc metadata
```

pack into Request

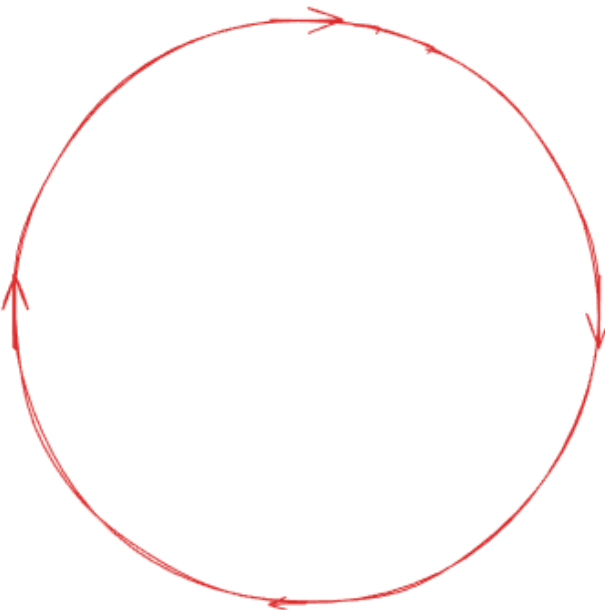


waiting queue



schedule

postprocess



forward pass

发动机回路

在流模式下，我们会在生成中间令牌时发送它们，但我们暂时忽略它。

接下来，我们将更详细地研究调度。

调度

推理引擎处理的工作负载主要有两种类型：

1. **预填充请求** – 向前传递所有提示令牌。这些通常是**计算受限**的（阈值取决于硬件和提示长度）。最后，我们从最终代币位置的概率分布中对单个代币进行采样。
2. **解码请求** – 仅向前传递最新的令牌。所有早期的 KV 向量都已缓存。这些是**内存带宽受限**的，因为我们仍然需要加载所有 LLM 权重（和 KV 缓存）来计算一个令牌。

在**基准测试部分**，我们将分析所谓的 GPU 性能车顶线模型。这将更详细地介绍预填充/解码性能配置文件。

V1 调度程序可以在同一步骤中混合两种类型的请求，这要归功于更智能的设计选择。相比之下，V0 引擎只能同时处理预填充或解码。

调度程序优先处理解码请求，即队列中已有的请求。对于每个这样的请求，它：**running**

1. 计算要生成的新令牌数（由于推测解码和异步调度，并不总是 1 个 – 稍后会详细介绍）。
2. 调用 KV 缓存管理器的函数（详情如下）。**allocate_slots**
3. 通过从步骤 1 中减去令牌数来更新令牌预算。

之后，它处理来自队列的预填充请求，它：**waiting**

1. 检索计算块的数量（如果禁用前缀缓存，则返回 0 – 我们稍后会介绍）。
2. 调用 KV 缓存管理器的函数。**allocate_slots**
3. 将请求从等待中弹出，并将其移至正在运行，将其状态设置为 **RUNNING**。
4. 更新令牌预算。

现在让我们看看它的作用：**allocate_slots**

1. **计算块数** – 确定必须分配多少个新的 KV 缓存块 $()$ 。默认情况下，每个区块存储 16 个代币。例如，如果预填充请求有 17 个新代币，我们需要区块。 **$n \text{ ceil}(17/16) = 2$**
2. **检查可用性** – 如果经理的池中没有足够的区块，请提前退出。根据是解码请求还是预填充请求，引擎可能会尝试通过逐出低优先级请求（调用将 KV 块返回到块池）来重新

计算抢占 (V0 中支持交换抢占), 或者它可能会跳过调度并继续执行。

`kv_cache_manager.free`

3. 分配块 - 通过 KV 缓存管理器的协调器, 从块池 (前面提到的双链表) 中获取第一个块。存储到 , 将每个字典映射到其 KV 缓存块列表。

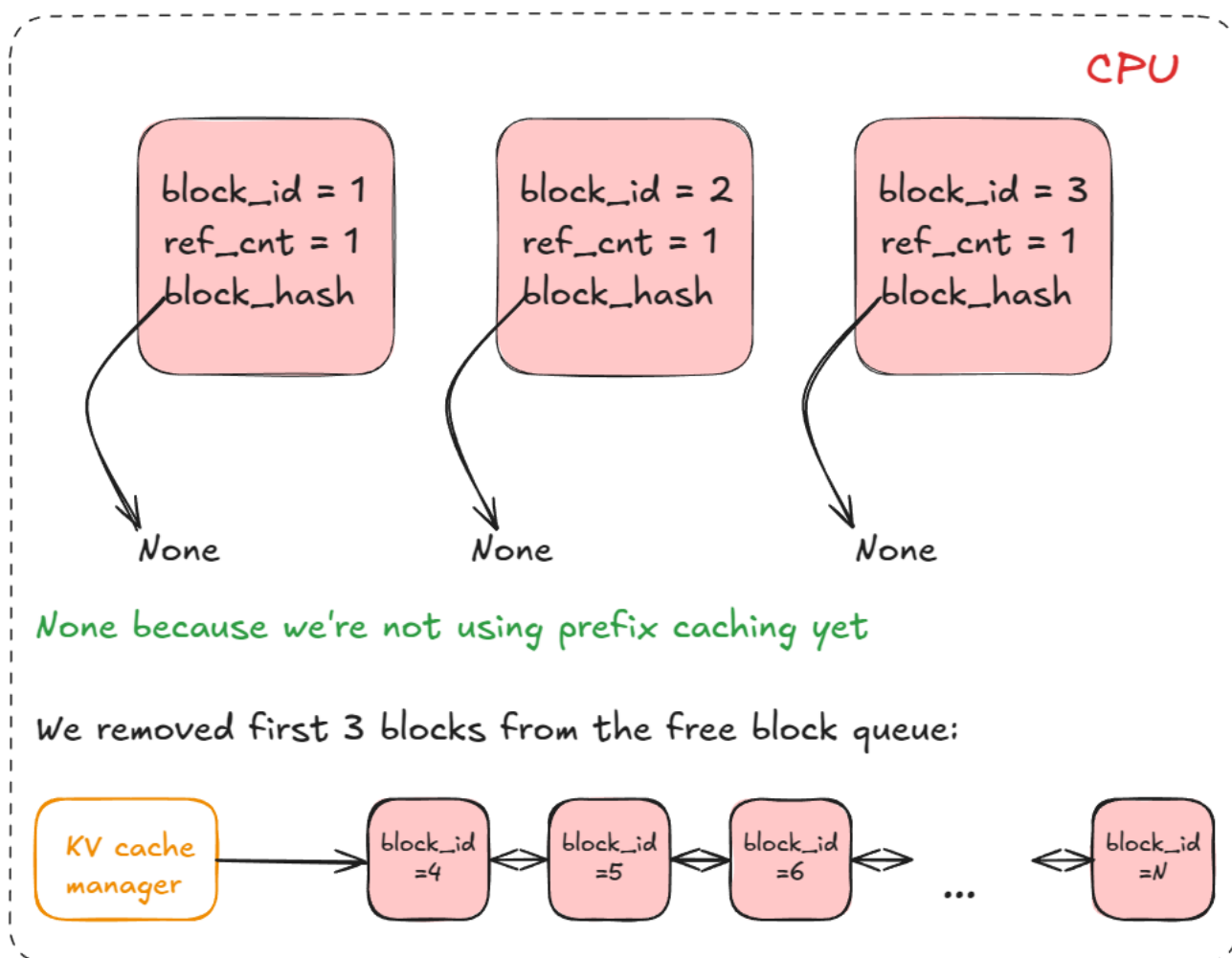
`n free_block_queue req_to_blocks request_id`

Example:

`prompt_token_ids = [1,2,3,4,5,6,7,8,9,10]`

`block_size = 4`

$\Rightarrow$ we need $\text{ceil}(10/4) = 3$ KV cache blocks!



KV 缓存块列表

我们终于准备好进行前进传球了!

跑向前传球

我们调用模型执行器的 , 它委托给 , 而 , 后者又委托给模型运行器。

`execute_model Worker`

以下是主要步骤:

1. **更新状态** – 修剪来自 ;更新 misc fwd 传递相关元数据 (例如, 每个请求的 KV 缓存块, 将用于索引到分页 KV 缓存存储器中)。 `input_batch`
2. **准备输入** – 从 CPU→GPU 复制缓冲区;计算位置;构建 (更多示例);构造注意力元数据。 `slot_mapping`
3. **前向传递** – 使用自定义分页 attn 内核运行模型。所有序列都被扁平化并连接成一个长“超级序列”。位置索引和注意力掩码确保每个序列只关注自己的标记, 从而实现连续批处理, 而无需右填充。
4. **收集最后一个标记状态** – 提取每个序列最终位置的隐藏状态并计算 logits。
5. **Sample** – 采样配置 (greedy、temperature、top-p、top-k 等) 规定的计算 logits 中的样本令牌。

前向传递步骤本身有两种执行模式:

1. **Eager 模式** – 启用 Eager 执行时运行标准 PyTorch 前向传递。
2. **“捕获”模式** – 在未强制执行 eager 时执行/重放预捕获的 CUDA 图 (请记住, 我们在引擎构建期间在初始化 KV 缓存过程中捕获了这些图)。

下面是一个具体示例, 应该清楚地表明连续批处理和分页注意力:

Example:

```
prompts = [
    "Hi, my name is",
    "Today is a beautiful summer day",
    "Hello there",
]
```

for the purpose of this example
i'm simplifying the tokenization,
but it's still consistent, e.g.
notice that "is" == 5, and
BOS token == 1

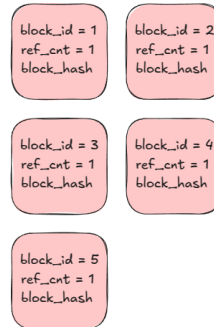
After tokenization:

```
[1,2,3,4,5]
[1,6,5,7,8,9,10]
[1,12,13]
```

block_size = 4

allocate_slots

gives us this:



CPU

continuous batching

converts the above into:

```
input_ids = [1,2,3,4,5,1,6,5,7,8,9,10,1,12,13]
positions = [0,1,2,3,4,0,1,2,3,4,5,6,0,1,2]
```

i.e. we flatten all of the sequences into a single "super sequence"

next up we compute slot_mapping:

```
slot_mapping = [4,5,6,7,8,12,13,14,15,16,17,18,20,21,22]
```

slot_mapping tells us where KVs
belonging to tokens from each sequence
go to inside the KV cache paged memory!

e.g. we can see that 2nd sequence goes to:
[12,13,14,15,16,17,18] - why?

GPU

initial state:



these belong to 2nd sequence

inside attn layer, during prefill, we compute KVs
and we store them in paged memory! so we end up with:

GPU

state after
1st fwd pass
(prefill)



KVs for 5 tokens
of 1st sequence

KVs for 7 tokens
of 2nd sequence

KVs for 3 tokens
of 3rd sequence

reshape_and_cache_flash

is responsible for updating the paged memory!

additional relevant attn metadata:
* query_start_loc = [0,5,12,15]
* seq_lens = [5,7,3]
* num_actual_tokens = 15

let's continue to decode now:

let's say we sampled the following tokens: [14,15,16] across the 3 sequences

continuous batching makes the above into:

```
input_ids = [1,2,3,4,5,14,15,16,1,6,5,7,8,9,10,1,12,13,16]
positions = [0,1,2,3,4,5,6,7,0,1,2,3,4,5,6,7,0,1,2,3]
slot_mapping = [4,5,6,7,8,9,12,13,14,15,16,17,18,19,20,21,22,23]
```

← appended 14,15,16 to sequences

← updated to reflect the new state

now we reuse the KVs from prefill step and only compute 1 new token per sequence!
all of this is possible thanks to the attn metadata (see right) and specialized attn kernels!

GPU

state after
2nd fwd pass
(decode)



KVs for 6 tokens
of 1st sequence

KVs for 8 tokens
of 2nd sequence

KVs for 4 tokens
of 3rd sequence

relevant attn metadata:

* query_start_loc = [0,1,2,3]
* seq_lens = [6,8,4]
* num_actual_tokens = 3

Note that it might not make sense from my explanation how attn metadata is used exactly,
those details are hidden in the attn kernels. We might cover that in one of the following posts!

前向传递：连续批处理和分页注意

有了基本的引擎流程，我们现在可以查看高级功能。

我们已经讨论了抢占、分页关注和连续批处理。

接下来，我们将深入探讨：

1. 分块预填充
2. 前缀缓存
3. 引导解码（通过语法约束的有限状态机）
4. 推测解码
5. 分解 P/D（预填充/解码）

分块预填充

分块预填充是一种通过将预填充步骤拆分为更小的块来处理长提示的技术。如果没有它，我们最终可能会得到一个非常长的请求，垄断一个引擎步骤，不允许其他预填充请求运行。这将推迟所有其他请求并增加其延迟。

例如，让每个块包含（=8）个标记，用“-”分隔的小写字母标记。长提示可能看起来像，其中是一个不完整的块（例如 2 toks）。执行完整的预填充将需要 ≥ 3 个引擎步骤（如果未在其中一个步骤中计划执行，则可能会发生>），并且只有在最后一个分块预填充步骤中，我们才会对新令牌进行采样。 

这是视觉上的相同示例：

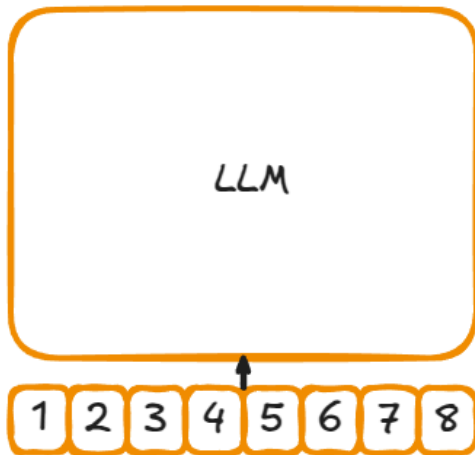
Example:

long_prefill_token_threshold = 8 toks

block_size = 4 toks

prompt_token_ids = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18]

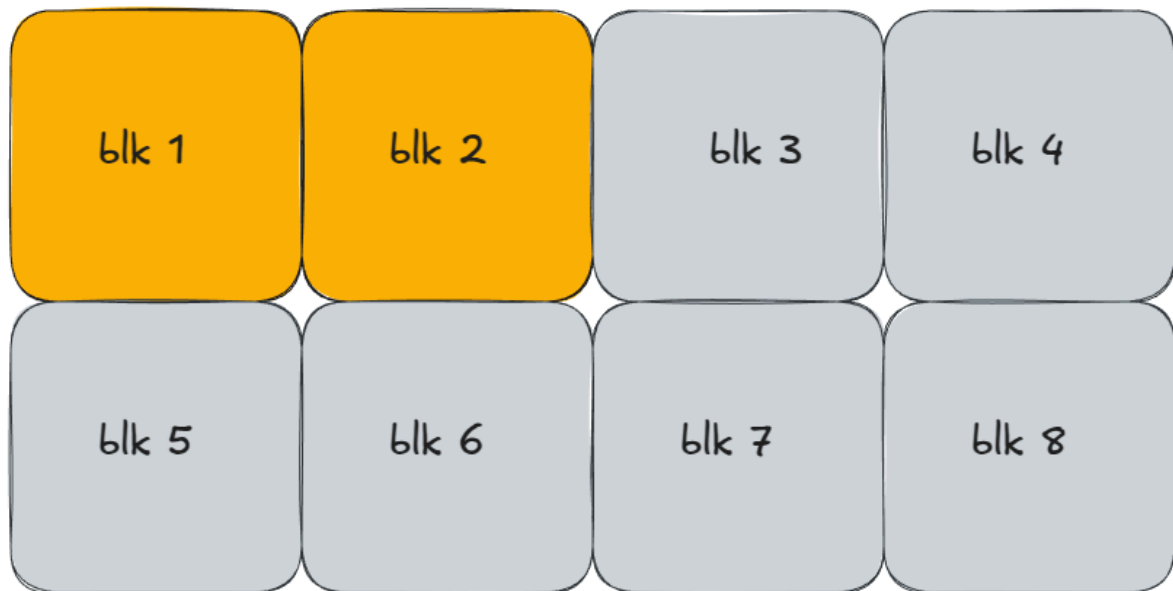
1st fwd pass



GPU

KV cache paged memory

after 1st fwd pass 2 blocks contain KVs (8 tokens)



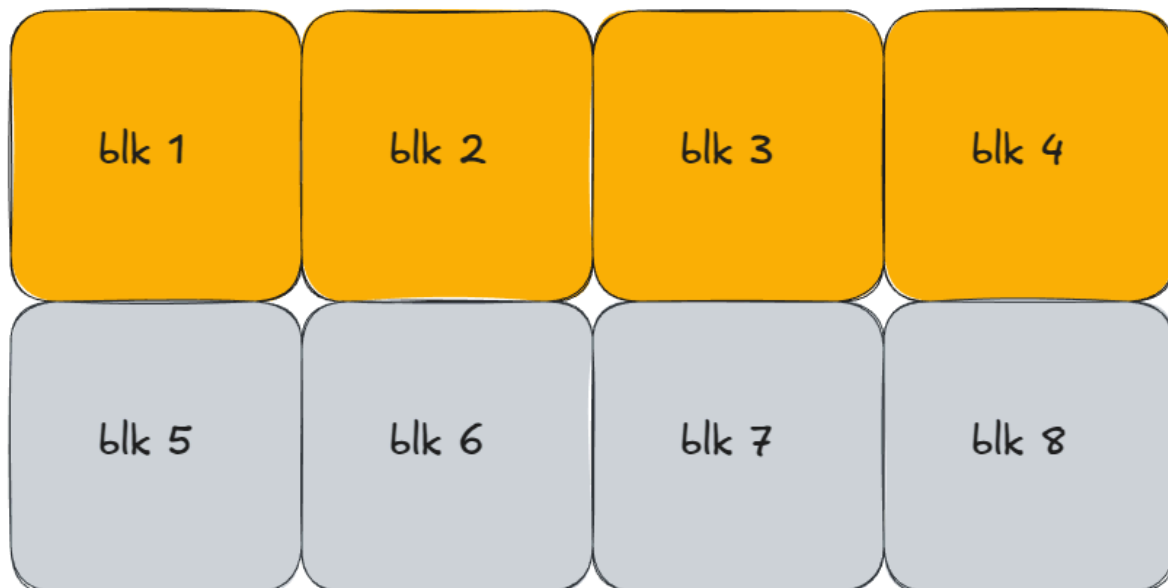
2nd fwd pass





GPU

after 2nd fwd pass 4 blocks contain KV (16 tokens)



实现很简单：限制每一步的新代币数量。如果请求的数字超过 `long_prefill_token_threshold`，请将其重置为该值。底层索引逻辑（前面描述）负责其余部分。

在 vLLM V1 中，通过设置为正整数来启用分块预填充。（从技术上讲，无论如何，它都可能发生，如果提示长度超过代币预算，我们会截断它并运行分块预填充。

`long_prefill_token_threshold`

前缀缓存

为了解释前缀缓存的工作原理，让我们以原始代码示例进行一些调整：

```
from vllm import LLM, SamplingParams

long_prefix = "<a piece of text that is encoded into more than block_size to

prompts = [
    "Hello, my name is",
    "The president of the United States is",
]

sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

def main():
    llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0")

    outputs = llm.generate(long_prefix + prompts[0], sampling_params)
    outputs = llm.generate(long_prefix + prompts[1], sampling_params)
```

```
if __name__ == "__main__":  
    main()
```

前缀缓存避免了重新计算多个提示在开始时共享的令牌 - 因此是前缀。

关键部分是：它被定义为任何长于 KV 缓存块（默认为 16 个令牌）的前缀。为了简化我们的示例，假设 has exactly length （其中）。
`long_prefix` `long_prefix` `n x`
`block_size` `n ≥ 1`

也就是说，它与区块边界完美对齐——否则我们将不得不重新计算令牌，因为我们无法缓存不完整的区块。
`long_prefix_len % block_size`

如果没有前缀缓存，每次我们处理具有相同的新请求时，我们都会重新计算所有令牌。

`long_prefix` `n x` `block_size`

使用前缀缓存时，这些令牌会计算一次（它们的 KV 存储在 KV 缓存分页内存中），然后重复使用，因此只有新的提示令牌需要处理。这加快了预填充请求的速度（尽管它对解码没有帮助）。

这在 vLLM 中是如何工作的？

在第一次调用期间，在调度阶段，引擎调用：

`generate` `kv_cache_manager.get_computed_blocks` `hash_request_tokens`

1. 此函数将 拆分为 16 个令牌块。`long_prefix + prompts[0]`
2. 对于每个完整的块，它都会计算一个哈希值（使用内置哈希值或 SHA-256，后者速度较慢，但冲突较少）。哈希值结合了前一个区块的哈希值、当前代币和可选元数据。

可选元数据包括：MM 哈希、LoRA ID、缓存盐（注入到第一个块的哈希中，确保只有具有此缓存盐的请求才能重用块）。

3. 每个结果都存储为包含哈希及其令牌 ID 的对象。我们返回一个区块哈希列表。

`BlockHash`

该列表存储在 中。`self.req_to_block_hashes[request_id]`

接下来，引擎调用以检查这些哈希值中是否已存在于。在第一个请求中，未找到任何命中。

`find_longest_cache_hit` `cached_block_hash_to_block`

EXAMPLE:

block_size = 4

long_prefix = "Today is a nice and warm summer day!"

long_prefix_tokens = [1,2,3,4,5,6,7,8] (in reality token ids are pulled from a vocab table)

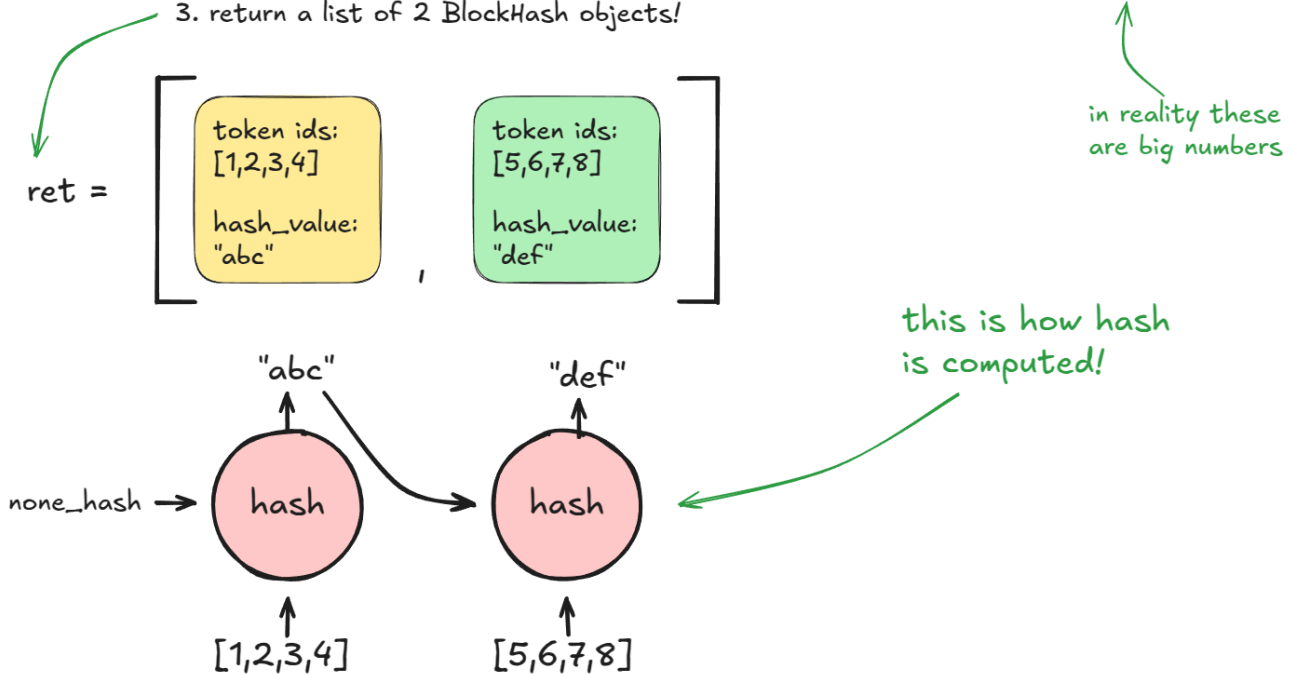
prompts = ["My name is", "His name is"]

prompts_tokens = [[9,10,2], [12,10,2]]

first call we pass in long_prefix + prompts[0] -> [1,2,3,4,5,6,7,8,9,10,2]

hash_request_tokens does the following:

1. splits input into [1,2,3,4], [5,6,7,8] and discards the last one ([9,10,2]) as incomplete
2. computes a hash for the first 2 chunks, let's say those are "abc" and "def"
3. return a list of 2 BlockHash objects!



find_longest_cache_hit returns []

because cached_block_hash_to_block = { }!

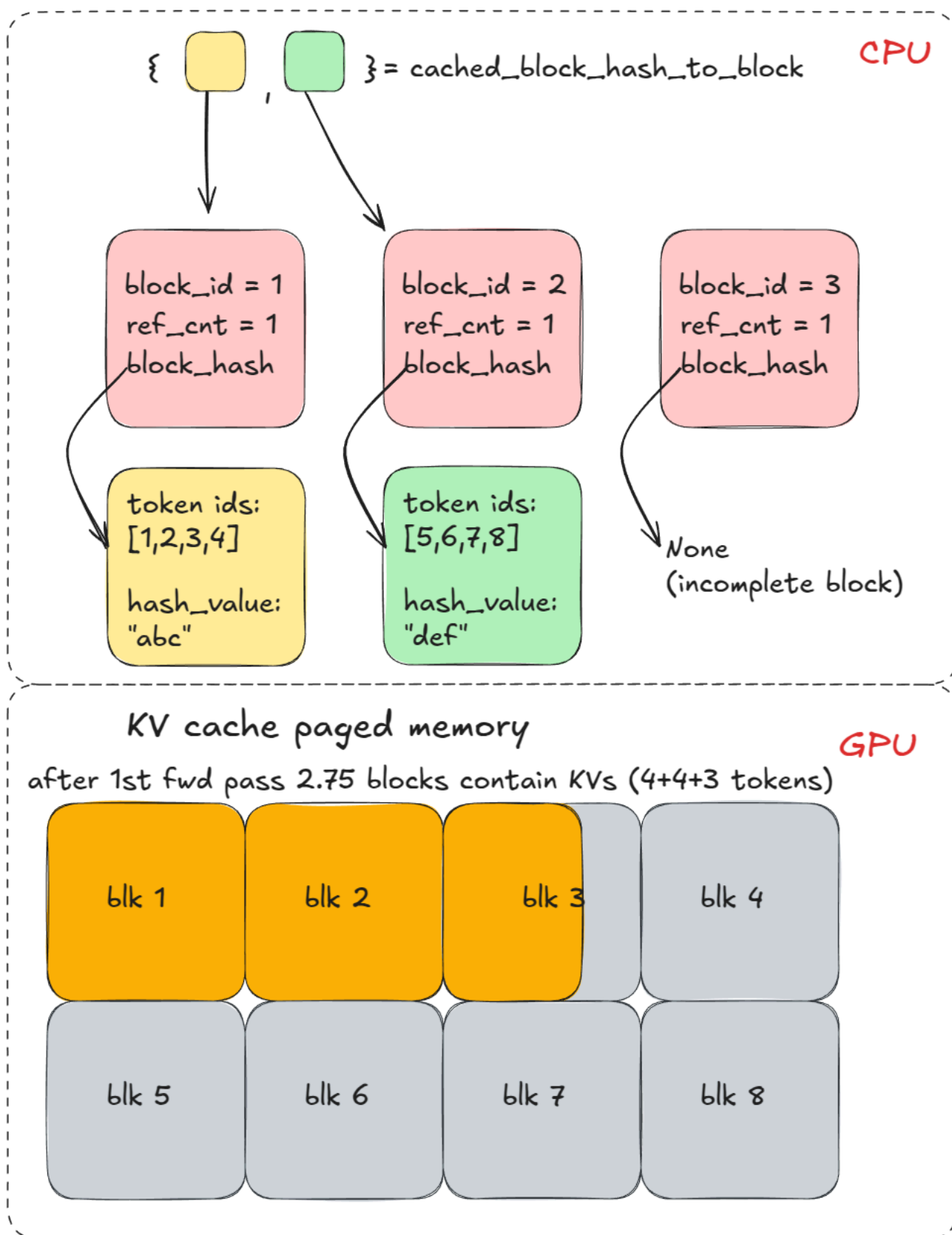
this makes sense we haven't yet done a fwd pass
so we haven't computed KVs for these 2 blocks!

然后我们调用 which 调用，它将新条目与分配的 KV 块相关联并将它们记录在

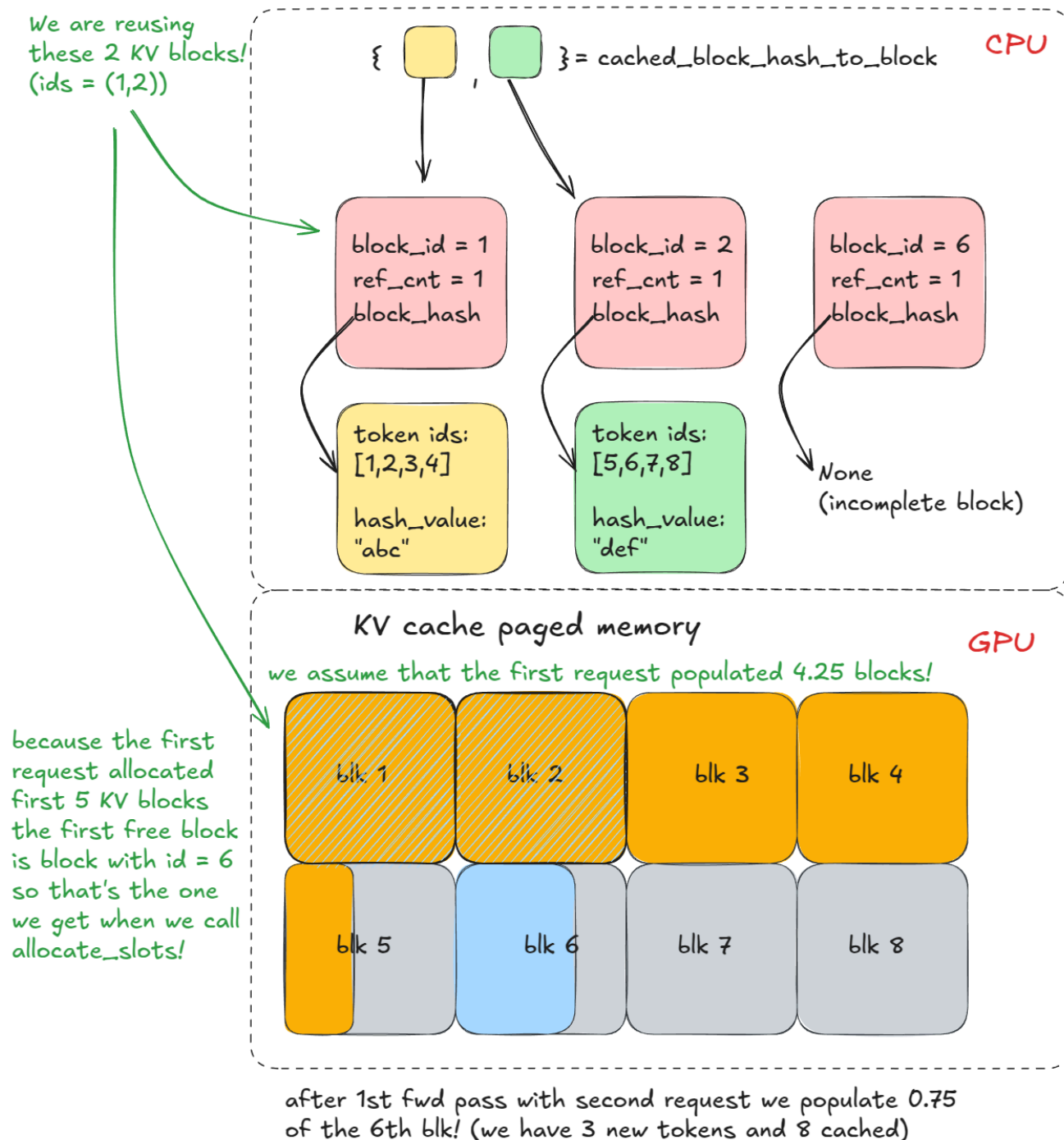
. allocate_slots coordinator.cache_blocks BlockHash cached_block_hash_to_block

之后，前向传递将在与我们上面分配的 KV 缓存块相对应的分页 KV 缓存内存中填充 KV。

经过许多引擎步骤后，它将分配更多的 KV 缓存块，但这对于我们的示例来说并不重要，因为前缀紧随其后。long_prefix



在具有相同前缀的第二次调用中，重复步骤 1-3，但现在查找所有块的匹配项（通过线性搜索）。引擎可以直接重用这些 KV 块。 `generate find_longest_cache_hit n`



如果原始请求仍然有效，则这些块的引用计数将递增（例如，到 2）。在此示例中，第一个请求已经完成，因此块被释放回池，并且它们的引用计数设置回 0。因为我们能够从我们知道的它们是有效的（KV 缓存管理器的逻辑是以这种方式设置的），所以我们只是再次将它们删除。 `cached_block_hash_to_block` `free_block_queue`



高级说明：

KV 缓存块仅在即将从（从左侧弹出）重新分配时才失效，并且我们发现该块仍然具有关联的哈希值并且存在于。此时，我们清除区块的哈希值并从 中删除其条目，确保它不能通过前缀缓存重用（至少对于旧前缀不能）。

`free_block_queue` `cached_block_hash_to_block` `cached_block_hash_to_block`

这就是前缀缓存的要点：不要重新计算你已经看到的前缀——只需重用它们的 KV 缓存！

如果您了解了这个示例，您也了解了分页注意力的工作原理。

默认情况下，前缀缓存处于启用状态。要禁用它： `enable_prefix_caching = False`

引导解码（FSM）

引导解码是一种技术，在每个解码步骤中，`logit` 都受到基于语法的有限状态机的约束。这确保只能对语法允许的标记进行采样。

这是一个强大的设置：你可以强制执行任何内容，从常规语法（Chomsky 类型 3，例如任意正则表达式模式）一直到上下文无关语法（类型 2，涵盖大多数编程语言）。

为了不那么抽象，让我们从最简单的示例开始，以我们之前的代码为基础：

```
from vllm import LLM, SamplingParams
from vllm.sampling_params import GuidedDecodingParams

prompts = [
    "This sucks",
    "The weather is beautiful",
]

guided_decoding_params = GuidedDecodingParams(choice=["Positive", "Negative"])
sampling_params = SamplingParams(guided_decoding=guided_decoding_params)

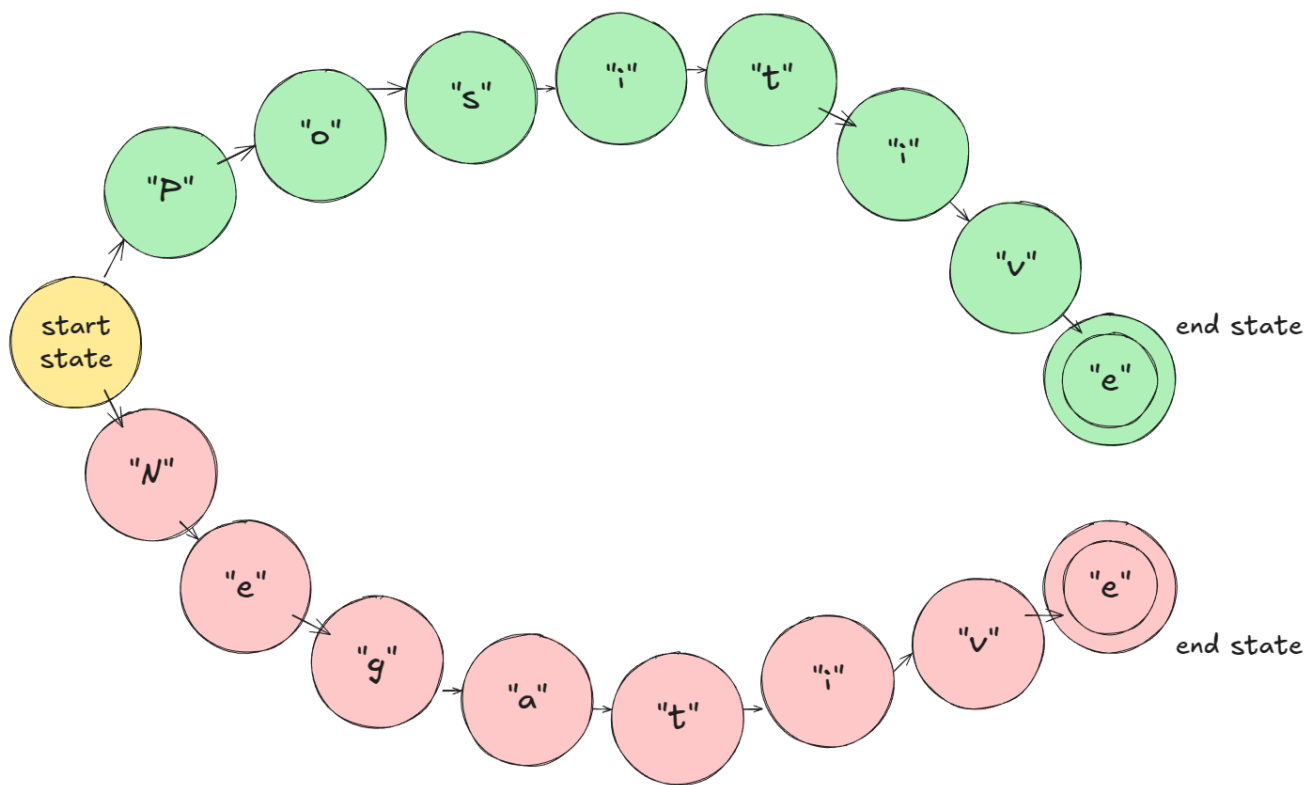
def main():
    llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0")

    outputs = llm.generate(prompts, sampling_params)

if __name__ == "__main__":
    main()
```

在我给出的玩具示例中（假设字符级标记化）：在预填充时，FSM 会屏蔽 `logits`，因此只有“P”或“N”是可行的。如果对“P”进行采样，则FSM移动到“正”分支；下一步只允许使用“O”，依此类推。

Extremely trivial FSM w/ 2 end states



玩具示例 FSM

这在 vLLM 中的工作原理：

1. 在 LLM 引擎构建中，创建了 `a`；它可以访问分词器并维护张量。
`StructuredOutputManager _grammar_bitmask`
2. 添加请求时，其状态设置为并选择后端编译器（例如，[\[7\]](#)；请注意，后端是第三方代码）。
`WAITING_FOR_FSM grammar_init xgrammar`
3. 此请求的语法是异步编译的。
4. 在调度期间，如果异步编译已完成，则状态将切换为 并被添加到 ；否则，它将被放置在下一个引擎步骤中重试。
`WAITING request_id structured_output_request_ids skipped_waiting_requests`
5. 调度循环后（仍在调度内部），如果有 FSM 请求，则要求后端准备/更新。
`StructuredOutputManager _grammar_bitmask`
6. 在前向传递产生 `logit` 后，`xgr_torch_compile` 的函数将位掩码扩展到词汇大小（32 倍扩展率，因为我们使用 32 位整数），并将不允许的 `logit` 屏蔽为 $-\infty$ 。
7. 对下一个令牌进行采样后，请求的 FSM 通过 推进。从视觉上看，我们移动到 FSM 图上的下一个状态。
`accept_tokens`

第 6 步值得进一步澄清。

如果 `vocab_size` 是单个整数; 它的二进制表示对允许 (“1”) 和不允许 (“0”) 的标记进行编码。例如, “101...001” 扩展到长度 32 的阵列; 0 的仓位将 logits 设置为 $-\infty$ 。对于较大的词汇表, 使用多个 32 位单词并相应地扩展/连接。后端 (例如,) 负责使用当前 FSM 状态生成这些位模式。 `vocab_size = 32 _grammar_bitmask [1, 0, 1, ..., 0, 0, 1] xgrammar`

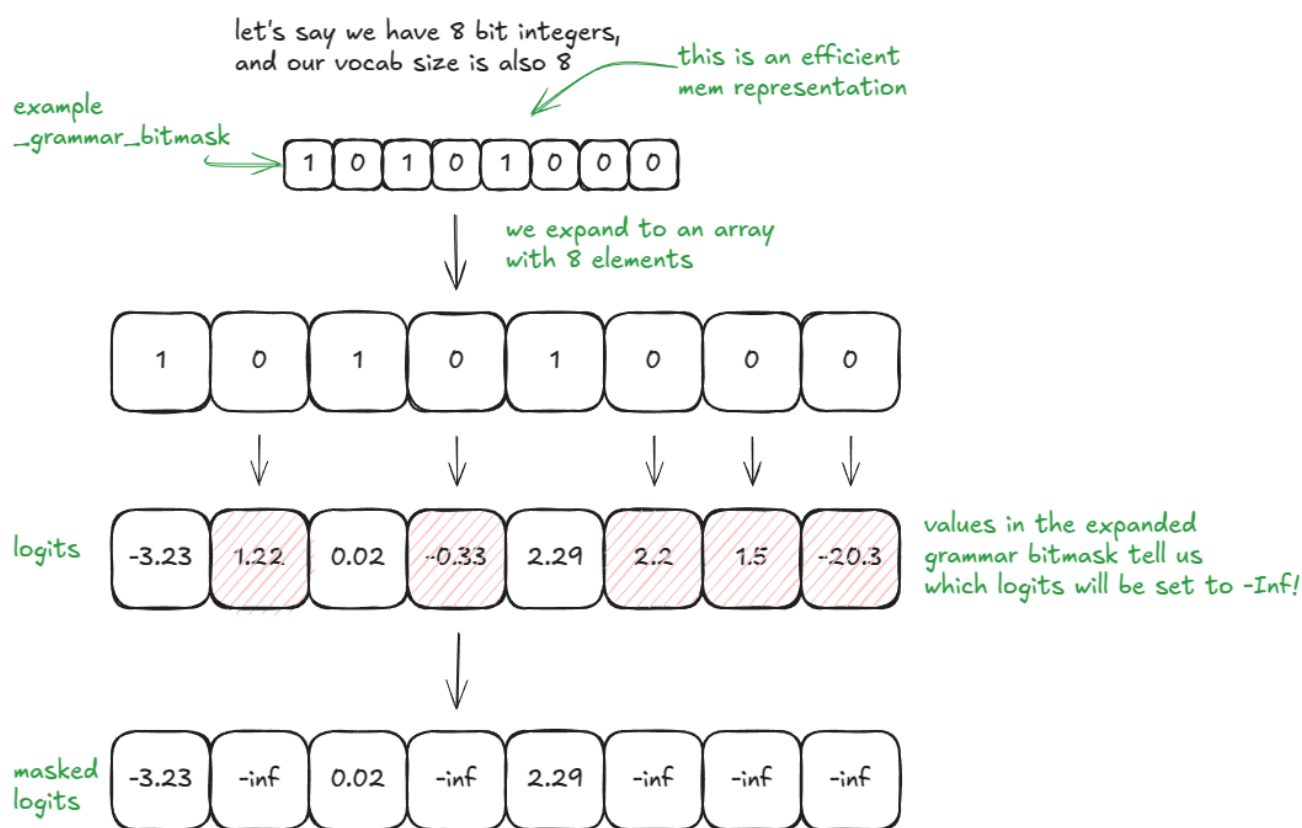


注意:

这里的大部分复杂性都隐藏在 `xgrammar` 等第三方库中。

这是一个更简单的示例, 具有 `vocab_size = 8` 位和 8 位整数 (对于那些喜欢我的视觉效果的人):

Example:



玩具示例

您可以通过传入所需的配置在 `vLLM` 中启用此功能。 `guided_decoding`

推测解码

在自回归生成中, 每个新代币都需要大 LM 的前向传递。这很昂贵——每一步都会重新加载并应用所有模型权重, 只是为了计算一个标记! (假设批量大小 == 1, 一般来说是 **B**)

推测解码 [8] 通过引入较小的草稿 LM 来加速这一进程。该草案以低廉的价格提出了代币。但我们最终不想从较小的模型中抽样——它只是为了猜测候选人的延续。大型模型仍然决

小红书

小红书号: 95420586205

定什么是有效的。 k

步骤如下：

1. **草稿**：在当前上下文上运行小模型并提出代币 k
2. **验证**：在上下文+草稿令牌上运行一次大模型。这会产生这些职位的概率加上一个额外的概率（这样我们就会得到候选人） $k \ k \ k+1$
3. **接受/拒绝**：从左到右在草稿令牌上： k
 - 如果大模型对草稿代币的概率 $\geq$ 草稿的概率，则接受
 - 否则，概率接受 $p_{\text{large}}(\text{token})/p_{\text{draft}}(\text{token})$
 - 在第一次拒绝时停止，或接受所有草稿令牌。 k
 - 如果所有草稿令牌都被接受，则还从大模型中“免费”采样额外的第 $-th$ 令牌（我们已经计算了该分布）。 $k \ (k+1)$
 - 如果存在拒绝，则在该位置创建一个新的重新平衡分布（，将最小值限制为 0，归一化为总和为 1），并从中采样最后一个标记。 $p_{\text{large}} - p_{\text{draft}}$

为什么有效：尽管我们使用小模型来提出候选者，但接受/拒绝规则保证了在预期中，序列的分布完全像我们从大模型中逐个标记采样一样。这意味着推测解码在统计上等同于标准的自回归解码——但速度可能要快得多，因为单个大模型传递可以让步代币。 $k+1$



注意：

我建议查看 [gpt-fast](#) 的简单实现，以及原始[论文](#)的数学细节和与完整模型采样的等效性证明。

vLLM V1 不支持 LLM 草案模型方法，而是实现了更快但不太准确的提案方案：n-gram、EAGLE [9] 和 Medusa [10]。

每句话：

1. **n-gram**：获取最后的代币；在序列中找到先前的匹配项；如果找到，则建议该匹配项之后的标记；否则递减窗口并重试至 $\text{prompt_lookup_max} \ k \ \text{prompt_lookup_min}$

当前实现在**第一次**匹配后返回令牌。引入新近度偏差并颠倒搜索方向感觉更自然？（即最后一场比赛） k

2. **Eagle**：对大型LM进行“模型手术”——保留嵌入和LM头，用轻量级MLP替换变压器堆栈；将其微调为廉价草稿

3. **美杜莎**：在大模型的顶部（LM 头之前嵌入）训练辅助线性头，并行预测下一个标记；使用这些头比运行单独的小型 LM 更有效地提出标记 **k**

以下是在 vLLM 中调用推测解码的方法，用作草稿方法：**ngram**

```
from vllm import LLM, SamplingParams

prompts = [
    "Hello, my name is",
    "The president of the United States is",
]

sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

speculative_config={
    "method": "ngram",
    "prompt_lookup_max": 5,
    "prompt_lookup_min": 3,
    "num_speculative_tokens": 3,
}

def main():
    llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0", speculative_config=speculative_config)

    outputs = llm.generate(prompts, sampling_params)

if __name__ == "__main__":
    main()
```

这在 vLLM 中是如何工作的？

设置（发动机制造期间）：

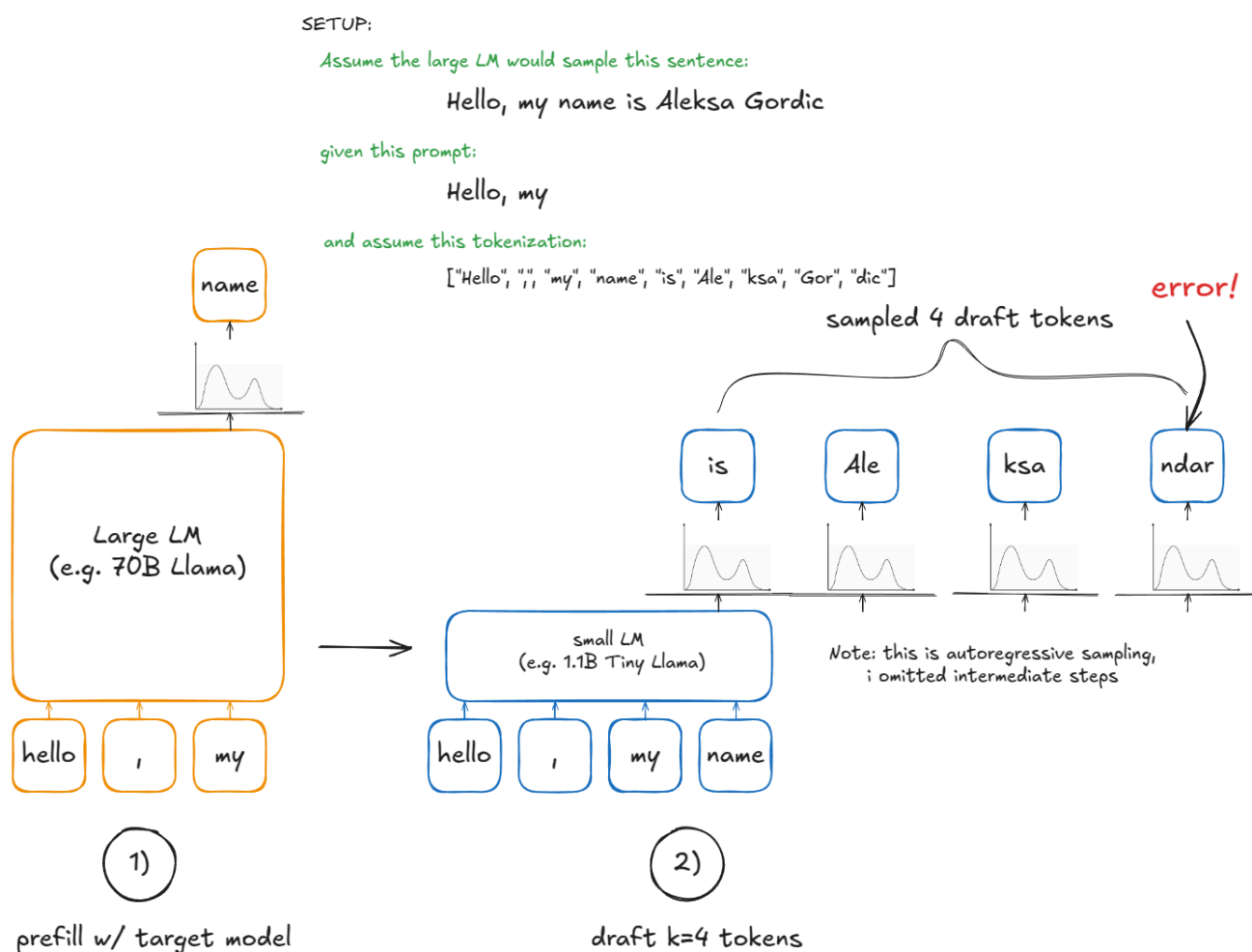
1. 初始化设备：创建 **a**（草稿模型，例如 **tinyllama**）和 **a**（其中的一部分用 Triton 编写）。
drafter NgramProposer rejection_sampler
2. 加载模型：加载拔模模型权重（n-gram 无运算）。

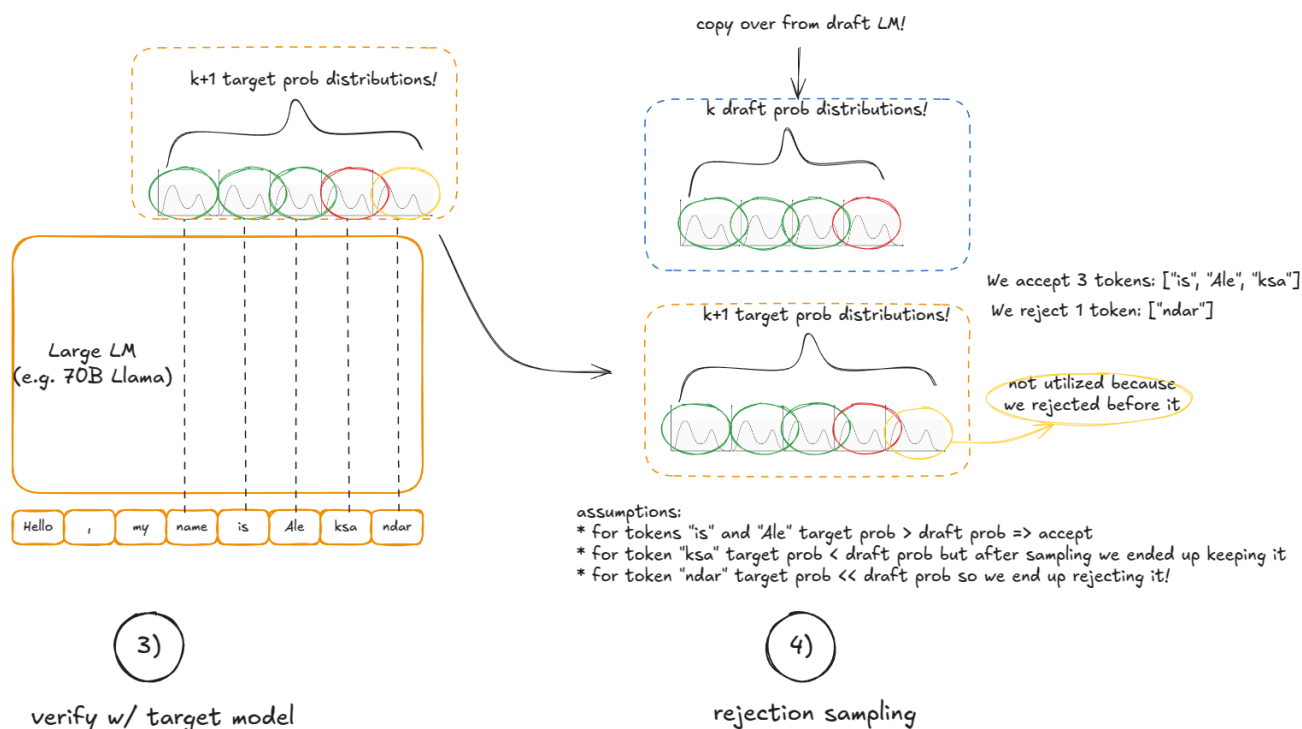
之后在 **generate** 函数中（假设我们收到一个全新的请求）：

1. 使用大型模型运行常规预填充步骤。
2. 在前向传递和标准采样之后，调用草稿模型中的草稿令牌。
propose_draft_token_ids(k) k
3. 将它们存储在（更新请求元数据）中。**request.spec_token_ids**

4. 在下一个引擎步骤中，当请求在运行队列中时，添加到“新令牌”计数中，以便为 fwd 传递保留足够的 KV 块。`len(request.spec_token_ids) allocate_slots`
5. 复制到表单（上下文 + 草稿）令牌。
`spec_token_ids input_batch.token_ids_cpu`
6. 通过计算元数据（这会从 、准备 logits 等复制标记），然后对草稿标记运行大模型前向传递。`_calc_spec_decode_metadata input_batch.token_ids_cpu`
7. 不要从 logit 进行常规采样，而是使用 从 左到右接受/拒绝 并生成
`.rejection_sampler output_token_ids`
8. 重复步骤 2-7，直到满足停止条件。

内部化这一点的最佳方法是启动调试器并单步执行代码库，但本节希望能让您体验一下。这也是：





We end up feeding ["Hello", ",", "my", "name", "is", "Ale", "ksa"] into the drafter - and the loop continues!

分类市盈率

我之前已经暗示过分解 P/D（预填充/解码）背后的动机。

预填充和解码具有非常不同的性能配置文件（计算绑定与内存带宽绑定），因此将它们的执行分开是一个明智的设计。它提供了对延迟的更严格控制——包括（第一个令牌的时间）和（令牌间延迟）——在**基准测试**部分有更多关于这一点。TFTT ITL

在实践中，我们运行 vLLM 预填充实例和 vLLM 解码实例，并根据实时请求组合自动扩展它们。预填充工作程序将 KV 写入专用的 KV 缓存服务；解码工作程序从中读取。这将长而突发的预填充与稳定的、对延迟敏感的解码隔离开来。NM

这在 vLLM 中是如何工作的？

为清楚起见，下面的示例依赖于，一个用于说明机制的调试连接器实现。

SharedStorageConnector

连接器是 vLLM 的抽象，用于处理实例之间的 KV 交换。连接器接口尚未稳定，计划进行一些近期改进，其中将涉及更改，其中一些可能会损坏。

我们启动 2 个 vLLM 实例（GPU 0 用于预填充，GPU 1 用于解码），然后在它们之间传输 KV 缓存：

```
import os
import time
```

```

from multiprocessing import Event, Process
import multiprocessing as mp

from vllm import LLM, SamplingParams
from vllm.config import KVTransferConfig

prompts = [
    "Hello, my name is",
    "The president of the United States is",
]

def run_prefill(prefill_done):
    os.environ["CUDA_VISIBLE_DEVICES"] = "0"

    sampling_params = SamplingParams(temperature=0, top_p=0.95, max_tokens=1)

    ktc=KVTransferConfig(
        kv_connector="SharedStorageConnector",
        kv_role="kv_both",
        kv_connector_extra_config={"shared_storage_path": "local_storage"},
    )

    llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0", kv_transfer_config=ktc)
    llm.generate(prompts, sampling_params)

    prefill_done.set() # notify decode instance that KV cache is ready

    # To keep the prefill node running in case the decode node is not done;
    # otherwise, the script might exit prematurely, causing incomplete decoding
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("Script stopped by user.")

def run_decode(prefill_done):
    os.environ["CUDA_VISIBLE_DEVICES"] = "1"

    sampling_params = SamplingParams(temperature=0, top_p=0.95)

    ktc=KVTransferConfig(
        kv_connector="SharedStorageConnector",
        kv_role="kv_both",
        kv_connector_extra_config={"shared_storage_path": "local_storage"},
    )

    llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0", kv_transfer_config=ktc)

    prefill_done.wait() # block waiting for KV cache from prefill instance

```

```
# Internally it'll first fetch KV cache before starting the decoding loop
outputs = llm.generate(prompts, sampling_params)

if __name__ == "__main__":
    prefill_done = Event()
    prefill_process = Process(target=run_prefill, args=(prefill_done,))
    decode_process = Process(target=run_decode, args=(prefill_done,))

    prefill_process.start()
    decode_process.start()

    decode_process.join()
    prefill_process.terminate()
```



注意：

我还尝试过 [11]，这是最快的生产就绪连接器（使用 NVIDIA 的 NIXL 作为后端），但它仍然处于最前沿，我遇到了一些错误。由于它的大部分复杂性都存在于外部存储库中，因此是解释的更好选择。 `LMCache SharedStorageConnector`

以下是 vLLM 中的步骤：

1. **实例化** – 在引擎构造期间，连接器在两个位置创建：
 - 在 worker 的 init 设备过程中（在 init worker 分布式环境功能下），角色为“worker”。
 - 在调度程序构造函数中，具有“调度器”角色。
2. **缓存查找** – 当调度程序处理来自队列的预填充请求时（在本地前缀缓存检查之后），它会调用连接器的 `waiting`。这将检查 KV 缓存服务器中的外部缓存令牌。预填充始终在此处看到 0；解码可能有缓存命中。在调用 `allocate_slots` 之前，将结果添加到本地计数中。
3. **状态更新** – 然后调度程序调用 `connector.update_state_after_alloc`，它记录具有缓存的请求（预填充无作）。
4. **元构建** – 在调度结束时，调度程序调用： `meta = connector.build_connector_meta`
 - Prefill 添加所有请求（以上传 KV）。 `is_store=True`
 - Decode 添加请求（以获取 KV）。 `is_store=False`
5. **上下文管理器** – 在前向传递之前，引擎进入 KV 连接器上下文管理器：
 - 在 Enter 时： `connector.start_load_kv` 被调用。对于解码，这会从外部服务器加载 KV 并将其注入分页内存中。对于预填充，这是一个无作的。

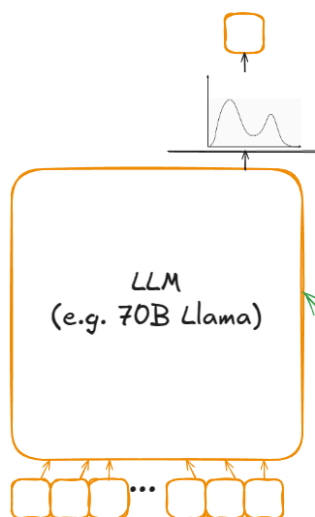
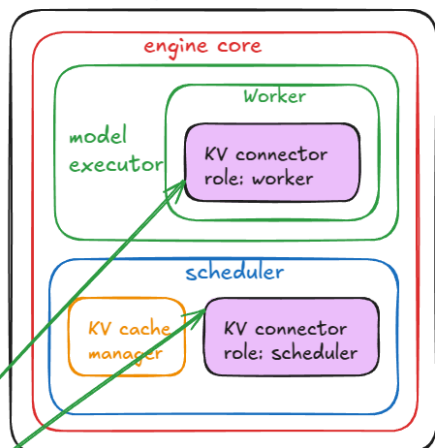
- 在退出时：被调用。对于预填充，这会阻止，直到 KV 上传到外部服务器。对于解码，这是一个无作的。`kv_connector.wait_for_save`

这是一个视觉示例：

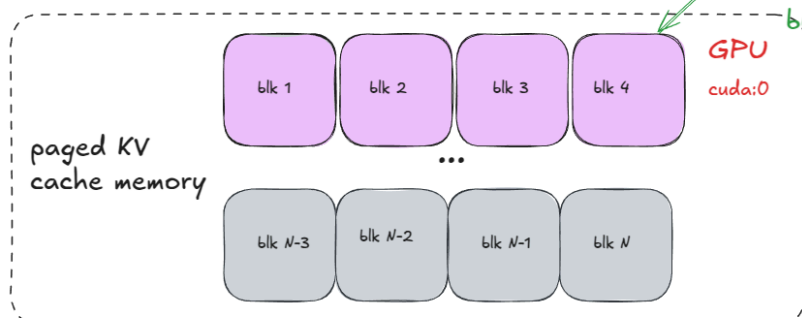
Note: simplified Engine Core to focus on P/D relevant bits

engine core client

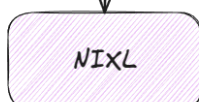
Prefill instance (cuda:0)



step1: compute KVs on prefill instance by doing a fwd pass

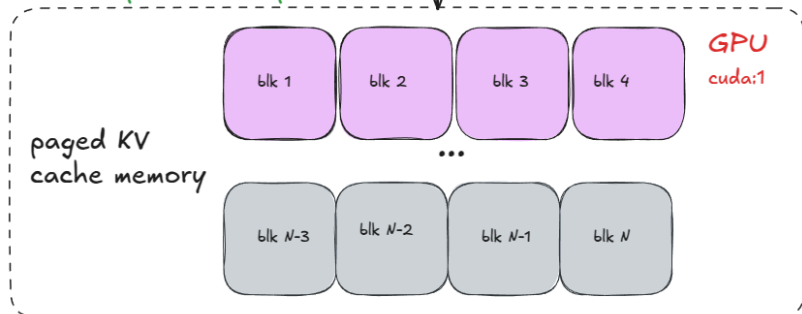


step2: KV connector (scheduler) delegates to KV connector (worker) who starts the transfer once prefill fwd pass is completed



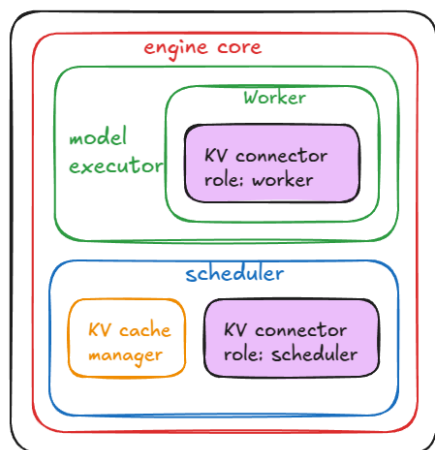
KVs from prefill instance (cuda:0) are copied over to decode instance (cuda:1) using NIXL backend

step3:



engine core client

Decode instance (cuda:1)



step4: decode instance reuses KVs from the prefill instance and computes (paged attn) the next token

分类市盈率



附加说明：

- 因为“外部服务器”只是一个本地文件系统。 `SharedStorageConnector`
- 根据配置的不同，KV 传输也可以逐层完成（在每个注意力层之前/之后）。
- Decode 在其请求的第一步仅加载一次外部 KV；之后，它在本地计算/存储。

从 UniprocExecutor 到 MultiProcExecutor

有了核心技术，我们现在可以讨论扩大规模了。

假设您的模型权重不再适合单个 GPU 的 VRAM。

第一种选择是使用张量并行性（例如，）跨同一节点上的多个 GPU 对模型进行分片（例如）。如果模型仍然不适合，下一步是跨节点的管道并行性。 `TP=8`

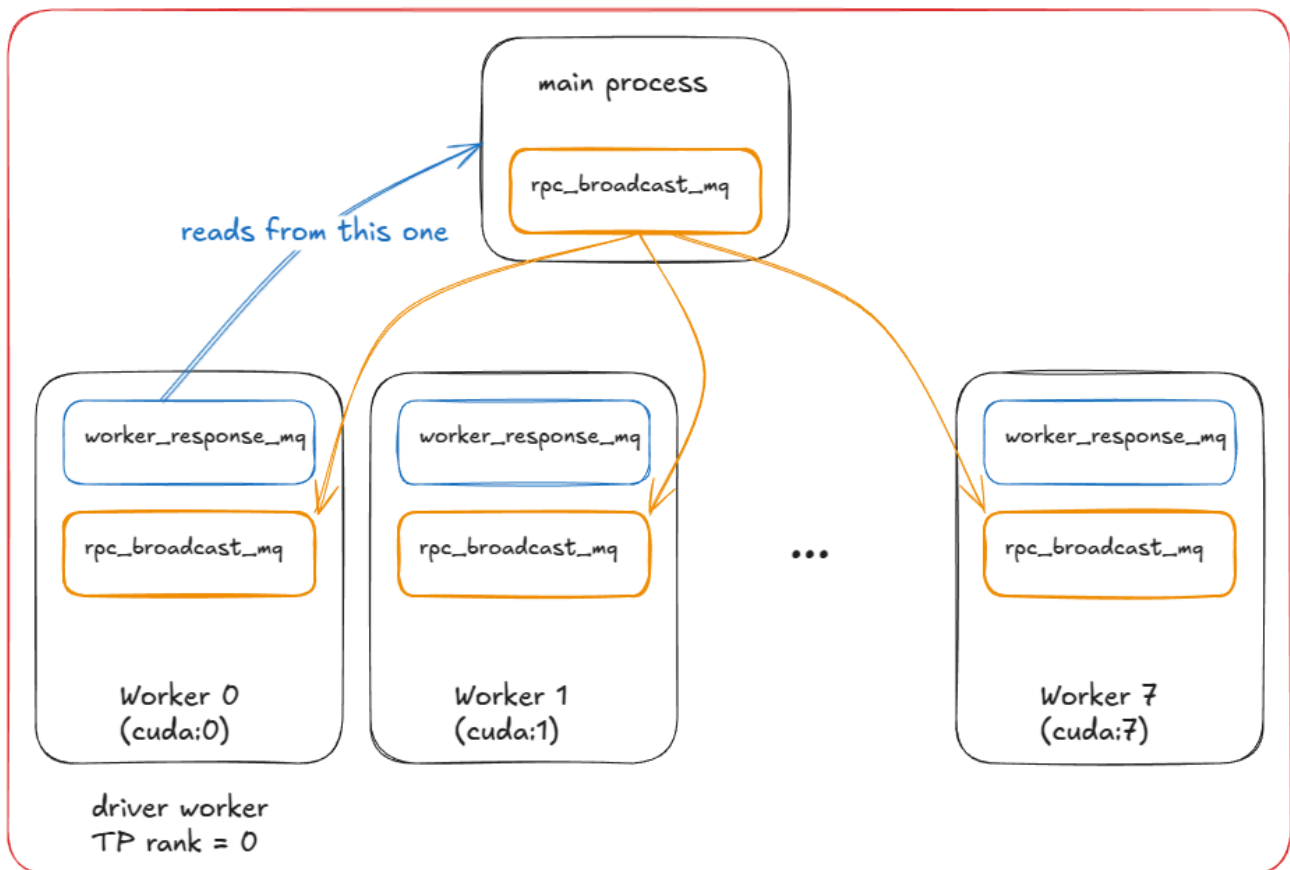


笔记：

- 节点内带宽明显高于节点间带宽，这就是为什么张量并行性（TP）通常优于管道并行性（PP）的原因。（PP 通信的数据也比 TP 少。）
- 我没有讨论专家并行性（EP），因为我们关注的是标准 Transformer 而不是 MoE，也不是序列并行性，因为 TP 和 PP 是实践中最常用的。

在这个阶段，我们需要多个 GPU 进程（工作线程）和一个编排层来协调它们。这正是所提供的。 `MultiProcExecutor`

MultiProcExecutor



TP=8 设置中的 MultiProcExecutor （驱动程序工作程序排名为 0）

这在 vLLM 中的工作原理：

1. `MultiProcExecutor` 初始化消息队列（在后台使用共享内存实现）。
`rpc_broadcast_mq`
2. 构造函数循环（例如 `for i in range(world_size):`）并通过 `WorkerProc.make_worker_process` 为每个等级生成一个守护进程。 `world_size TP=8 ⇒ world_size=8 WorkerProc.make_worker_process`
3. 对于每个工作程序，父级首先创建一个读取器和写入器管道。
4. 新进程运行，它实例化了一个工作程序（通过与 `UniprocExecutor` 中相同的“init device”、“load model”等）。 `WorkerProc.worker_main UniprocExecutor`
5. 每个工作人员确定它是司机（TP 组中的等级 0）还是常规工作人员。每个工作程序设置两个队列：
 - `rpc_broadcast_mq`（与家长共享）接收工作。
 - `worker_response_mq` 用于发回回复。
6. 在初始化期间，每个子级都通过管道将其句柄发送到父级。收到所有内容后，父级将取消阻止 - 这样就完成了协调。 `worker_response_mq`
7. 然后，工作人员进入一个繁忙的循环，阻塞。当工作项到达时，他们执行它（就像 `UniprocExecutor` 中一样，但现在使用特定于 TP/PP 的分区工作）。结果通过 `worker_response_mq` 发回。

`rpc_broadcast_mq.dequeue UniprocExecutor worker_response_mq.enqueue`

8. 在运行时，当请求到达时，将其排入所有子工作程序的队列（非阻塞）。然后，它等待指定的输出排名以收集最终结果。

`MultiProcExecutor rpc_broadcast_mq worker_response_mq.dequeue`

从引擎的角度来看，没有任何变化——所有这些多处理复杂性都通过调用模型执行器的抽象化。`execute_model`

- 在这种情况下：`execute_model`直接导致对工作人员的呼叫
`execute_model UniprocExecutor`
- 在以下情况下：`execute_model`间接导致通过以下方式对每个工作程序调用
`execute_model MultiProcExecutor rpc_broadcast_mq`

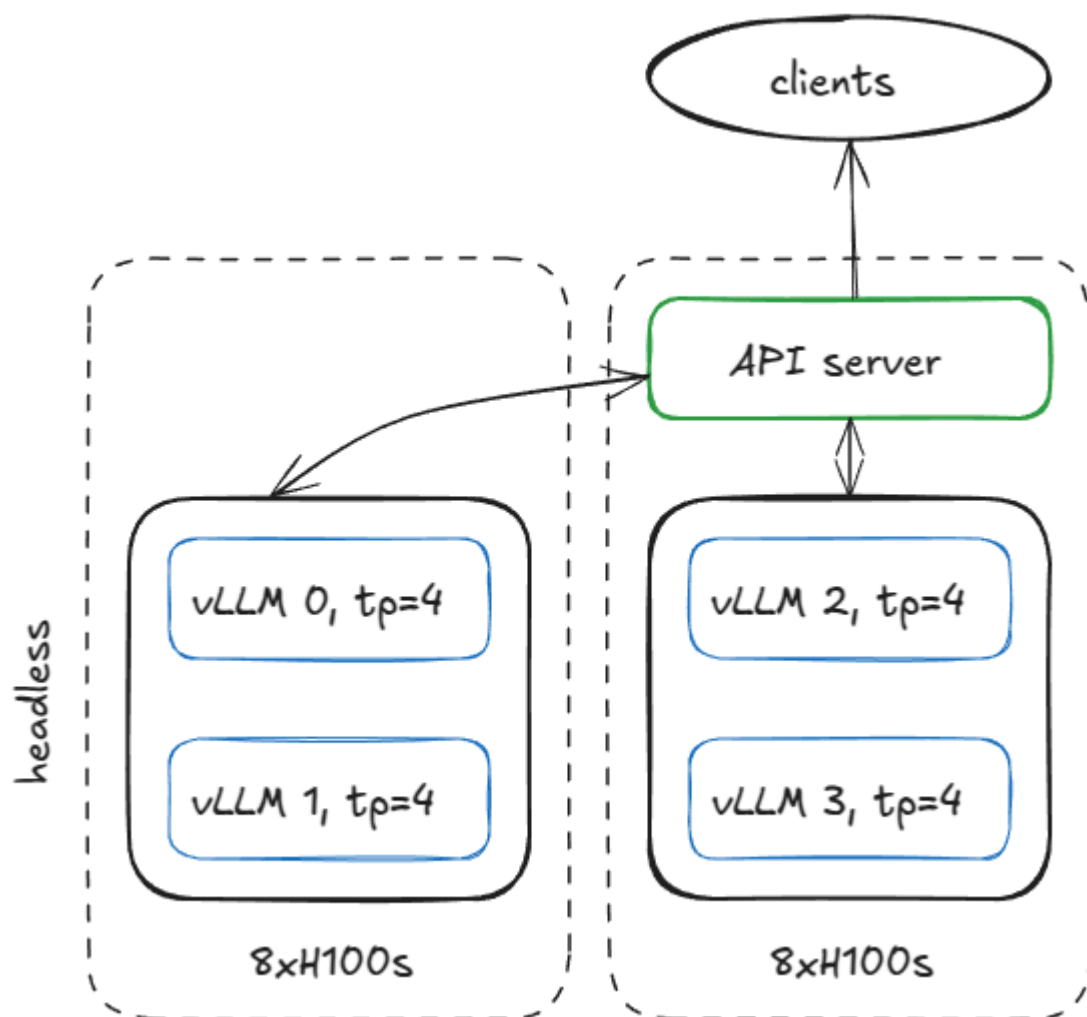
此时，我们可以使用相同的引擎接口运行资源允许的尽可能大的模型。

下一步是横向扩展：启用跨节点复制模型的数据并行性（），添加轻量级 DP 协调层，引入跨副本的负载平衡，并将一个或多个 API 服务器放在前面来处理传入流量。`DP > 1`

服务于 vLLM 的分布式系统

设置服务基础设施的方法有很多种，但为了保持具体，这里有一个例子：假设我们有两个 H100 节点，并希望在它们之间运行四个 vLLM 引擎。

如果模型需要，我们可以这样配置节点。`TP=4`



具有 2 个 8xH100 节点的服务器配置 (1 个无头服务器, 1 个 API 服务器)

在第一个节点上, 使用以下参数以无头模式 (无 API 服务器) 运行引擎:

```
vllm serve <model-name>
--tensor-parallel-size 4
--data-parallel-size 4
--data-parallel-size-local 2
--data-parallel-start-rank 0
--data-parallel-address <master-ip>
--data-parallel-rpc-port 13345
--headless
```

并在另一个节点上运行相同的命令, 只需进行一些调整:

- 不 `--headless`
- 修改DP起始等级

```
vllm serve <model-name>
--tensor-parallel-size 4
--data-parallel-size 4
--data-parallel-size-local 2
```

```
--data-parallel-start-rank 2
--data-parallel-address <master-ip>
--data-parallel-rpc-port 13345
```



注意：

这假设已配置网络，以便所有节点都可以到达指定的 IP 和端口。

这在 VLLM 中是如何工作的？

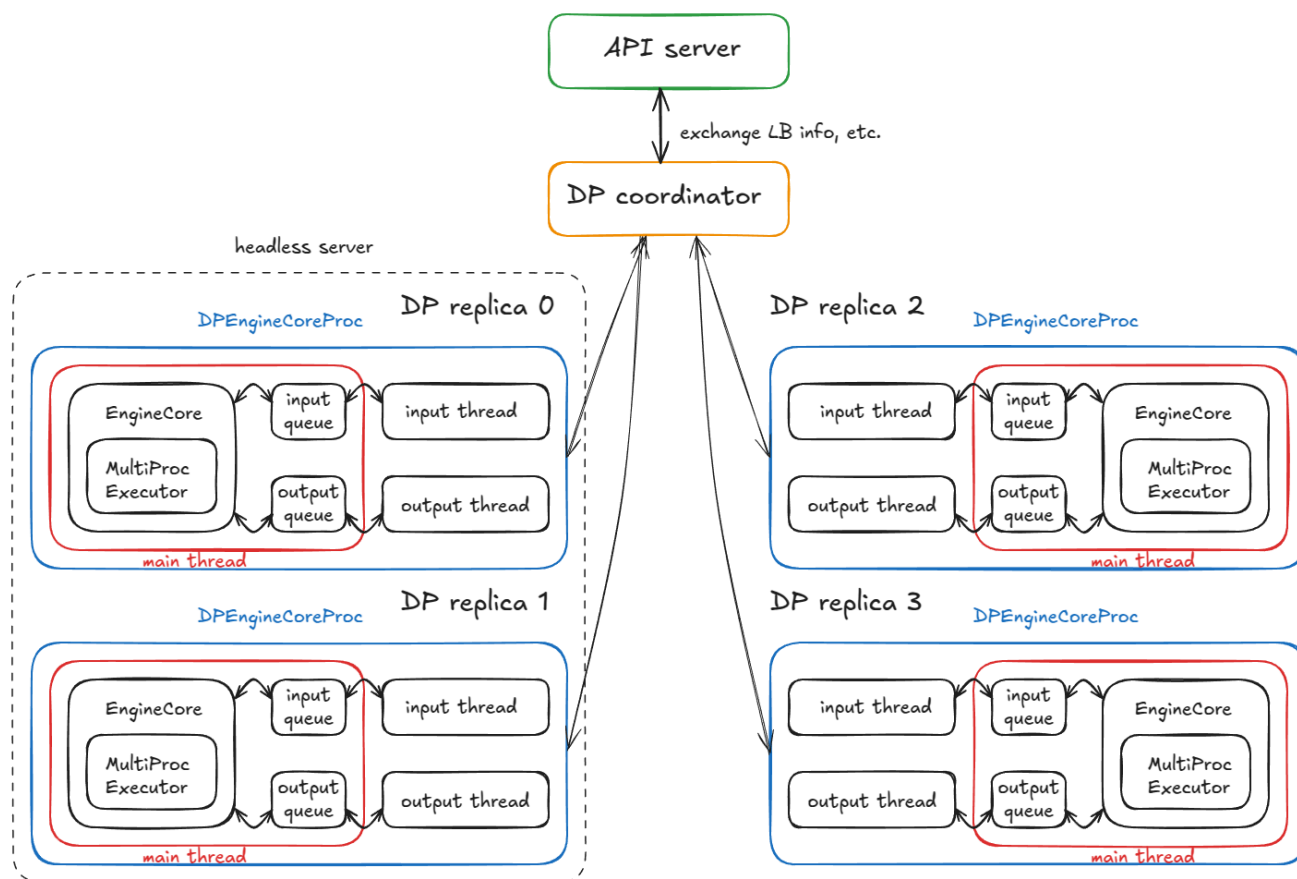
在无头服务器节点上

在无头节点上，a 启动 2 个进程（每个），每个进程运行 。这些函数中的每一个都创建一个（引擎核心），然后进入其繁忙循环。 `CoreEngineProcManager` `--data-parallel-size-local` `EngineCoreProc.run_engine_core` `DPEngineCoreProc`

`DPEngineCoreProc` 初始化其父级（子级），其中： `EngineCoreProc` `EngineCore`

1. 创建一个 `and`（）。 `input_queue` `output_queue` `queue.Queue`
2. 使用 ZMQ 套接字（异步消息传递库）与另一个节点上的前端执行初始握手，并接收协调地址信息。 `DEALER`
3. 初始化 DP 组（例如使用 NCCL 后端）。
4. 初始化 `with`（如前所述在 4 个 GPU 上）。 `EngineCore` `MultiProcExecutor` `TP=4`
5. 创建一个（）。 `ready_event` `threading.Event`
6. 启动运行的输入 `daemon` 线程（）。同样启动输出线程。 `threading.Thread` `process_input_sockets(..., ready_event)`
7. 仍然在主线程中，等待所有 4 个进程（跨越 2 个节点）中的所有输入线程完成协调握手，最终执行。 `ready_event` `ready_event.set()`
8. 解锁后，向前端发送一条“就绪”消息，其中包含元数据（例如，在分页 KV 高速缓存中可用）。 `num_gpu_blocks`
9. 然后，主线程、输入线程和输出线程进入各自的繁忙循环。

TL;DR: 我们最终有 4 个子进程（每个 DP 副本一个），每个子进程运行一个主线程、输入线程和输出线程。他们完成与 DP 协调器和前端的协调握手，然后每个进程的所有三个线程都在稳态繁忙循环中运行。



具有 4 个运行 4 个 DPEngineCoreProc 的 DP 副本的分布式系统

当前稳态:

- **输入线程** – 在输入套接字上阻塞，直到从 API 服务器路由请求;收到后，它会解码有效负载，通过 将工作项排入队列，然后返回套接字上的阻塞。

```
input_queue.put_nowait(...)
```

- **主线程** – 唤醒，将请求馈送到引擎;运行前向传递并将结果排队到。

```
input_queue.get(...) MultiProcExecutor output_queue
```

- **输出线程** – 唤醒，将结果发送回 API 服务器，然后恢复阻塞。

```
output_queue.get(...)
```

附加机制:

- **DP 波计数器** – 系统跟踪“波”;当所有引擎都空闲时，它们会静止，并且当新工作到达时计数器会递增（对于协调/指标很有用）。
- **控制消息** – API 服务器可以发送的不仅仅是推理请求（例如，中止和实用程序/控制 RPC）。
- **锁步的虚拟步骤** – 如果任何 DP 副本有工作，则所有副本都执行前进步骤;没有请求的副本执行虚拟步骤以参与所需的同步点（避免阻止活动副本）。

锁步澄清：这实际上仅适用于专家层形成 EP 或 TP 组而注意力层仍为 DP 的 MoE 模型。目前它总是使用 DP 完成 - 这只是因为“内置”非 MoE DP 的使用有限，因为您可以运行多个独立的 vLLM 并以正常方式在它们之间进行负载平衡。

现在是第二部分，API 服务器节点上会发生什么？

在 API 服务器节点上

我们实例化一个对象（LLM 引擎周围的 `asyncio` 包装器）。在内部，这会创建一个（数据并行、负载平衡、异步、多处理客户端）。`AsyncLLM DPLBAsyncMPCClient`

在 的父类中，函数运行并：`MPCClient launch_core_engines`

1. 创建用于启动握手的 ZMQ 地址（如无头节点上所示）。
2. 生成一个进程。`DPCoordinator`
3. 创建一个（与无头节点相同）。`CoreEngineProcManager`

在内部（子项），我们：`AsyncMPCClient MPCClient`

1. 创建一个 `()`。`outputs_queue asyncio.Queue`
2. 我们创建一个 `asyncio` 任务，它与所有 4 个输出线程进行通信（通过输出套接字）并写入 `.process_outputs_socket DPEngineCoreProc outputs_queue`
3. 随后，从该队列中读取另一个 `asyncio` 任务，最后向函数发送信息。
`output_handler AsyncLLM create_completion`

在内部，我们创建了一个与 DP 协调器通信的 `asyncio` 任务。

`DPLBAsyncMPCClient run_engine_stats_update_task`

DP 协调器在前端（API 服务器）和后端（引擎核心）之间进行调解。它：

- 定期将负载平衡信息（队列大小、等待/运行请求）发送到前端的 `.run_engine_stats_update_task`
- 通过动态更改引擎数量来处理来自前端的命令（仅适用于 Ray 后端）。
`SCALE_ELASTIC_EP`
- 将事件发送到后端（当由前端触发时）并报告波形状态更新。`START_DP_WAVE`

回顾一下，前端 `()` 运行几个 `asyncio` 任务（记住：并发，而不是并行）：`AsyncLLM`

- 一类任务通过路径处理输入请求（每个新的客户端请求都会生成一个新的 `asyncio` 任务）。`generate`

- 两个任务 (,) 处理来自底层引擎的输出消息。

`process_outputs_socket output_handler`

- 一项任务 () 保持与 DP 协调器的通信：发送波次触发器、轮询 LB 状态和处理动态扩展请求。 `run_engine_stats_update_task`

最后，主服务器进程创建一个 FastAPI 应用程序并挂载端点，例如 和 ，这些端点会公开、和其他端点。然后通过 Uvicorn 提供堆栈。

`OpenAIServingCompletion OpenAIServingChat /completion /chat/completion`

因此，将所有内容放在一起，这是完整的请求生命周期！

您从终端发送：

```
curl -X POST http://localhost:8000/v1/completions -H "Content-Type: application/json" -d '{
  "model": "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
  "prompt": "The capital of France is",
  "max_tokens": 50,
  "temperature": 0.7
}'
```

接下来会发生什么：

1. 请求命中 API 服务器上的路由。 `OpenAIServingCompletion create_completion`
2. 该函数异步标记提示，并准备元数据（请求 ID、采样参数、时间戳等）。
3. 然后它调用 ，它遵循与同步引擎相同的流程，最终调用 。
`AsyncLLM.generate DPAsyncMPCClient.add_request_async`
4. 这反过来又调用 ，它根据 DP 协调器的状态跨引擎进行负载平衡（选择得分最低/负载最低的引擎：）。 `get_core_engine_for_request score = len(waiting) * 4 + len(running)`
5. 请求将发送到所选引擎的 `.ADD input_socket`
6. 在那个引擎上：
 - 输入线程 - 取消阻止、解码来自输入套接字的数据，并将工作项放在主线程的上。 `input_queue`
 - 主线程 - 在 上解锁 ，将请求添加到引擎，并重复调用 ，将中间结果排队，直到满足停止条件。 `input_queue engine_core.step() output_queue`

提醒：调用调度器、模型执行器（反过来可以是！）等。我们已经看到了这一点！

`step() MultiProcExecutor`

- 输出线程 – 取消阻塞并通过输出套接字将结果发送回。 `output_queue`

7. 这些结果会触发输出的 `asyncio` 任务 (和), 这些任务将令牌传播回 FastAPI 的路由。 `AsyncLLM process_outputs_socket output_handler create_completion`

8. FastAPI 附加元数据 (完成原因、logprobs、使用信息等) 并通过 Uvicorn 返回到您的终端! `JSONResponse`

就这样, 你的完成又回来了——整个分布式机器隐藏在一个简单的命令后面! :)太有趣了!! `curl`



附加说明:

- 添加更多 API 服务器时, 负载均衡在作系统/套接字级别处理。从应用程序的角度来看, 没有什么重大变化——复杂性被隐藏了。
- 使用 Ray 作为 DP 后端, 您可以公开一个 URL 端点 (), 该终端节点可以自动扩展或缩减引擎副本的数量。 `/scale_elastic_ep`

基准测试和自动调整 - 延迟与吞吐量

到目前为止, 我们一直在分析“气体颗粒”——请求如何流经引擎/系统的内部结构。现在是时候缩小范围, 从整体上看待系统, 并问: 我们如何衡量推理系统的性能?

在最高级别, 有两个相互竞争的指标:

1. **延迟** – 从提交请求到返回令牌的时间
2. **吞吐量** – 系统每秒可以生成/处理的令牌/请求数

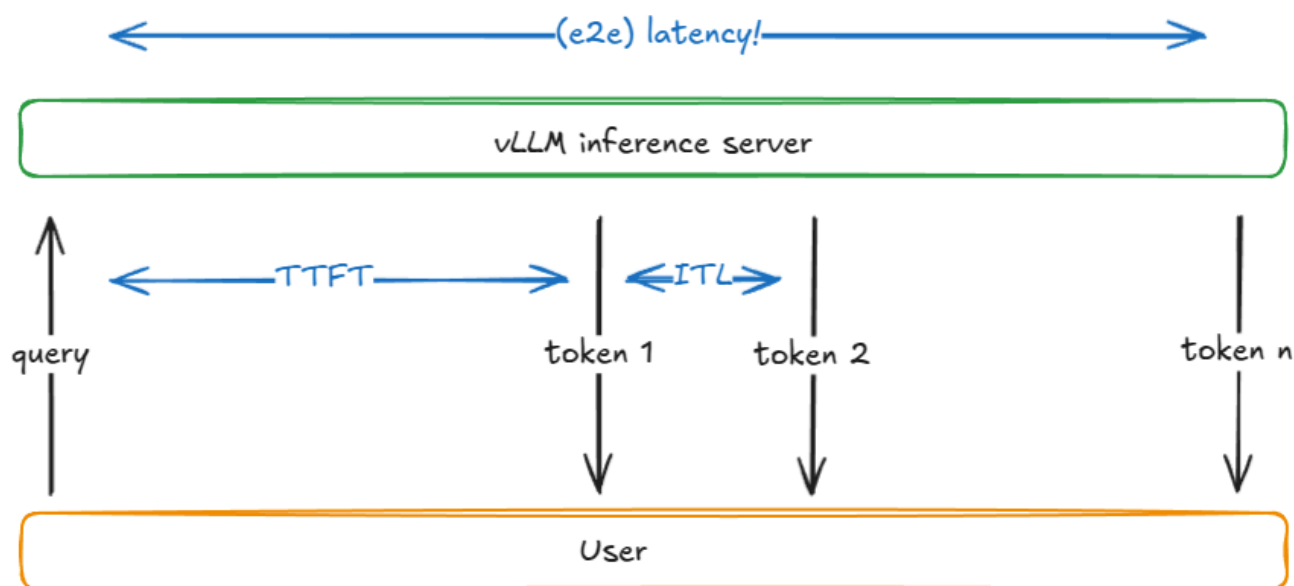
延迟对于用户正在等待响应的交互式应用程序来说最为重要。

吞吐量在离线工作负载中很重要, 例如用于训练前/训练后运行的合成数据生成、数据清理/处理, 以及一般情况下 - 任何类型的离线批量推理作业。

在解释延迟和吞吐量竞争的原因之前, 让我们定义一些常见的推理指标:

度量	定义
TTFT (第一个令牌的时间)	从请求提交到收到第一个输出令牌的时间

度量	定义
ITL (令牌间延迟)	两个连续标记之间的时间（例如，从标记 $i-1$ 到标记 i ）
TPOT (每个输出令牌的时间)	请求中所有输出令牌的平均 ITL
Latency / E2E (端到端延迟)	处理请求的总时间，即 TTFT + 所有 ITL 的总和，或等效于提交请求和接收最后一个输出令牌之间的时间
Throughput	每秒处理的令牌总数（输入、输出或两者）或每秒请求数
Goodput	满足服务级别目标（SLO）的吞吐量，例如最大 TTFT、TPOT 或 e2e 延迟。例如，仅计算来自满足这些 SLO 的请求的令牌



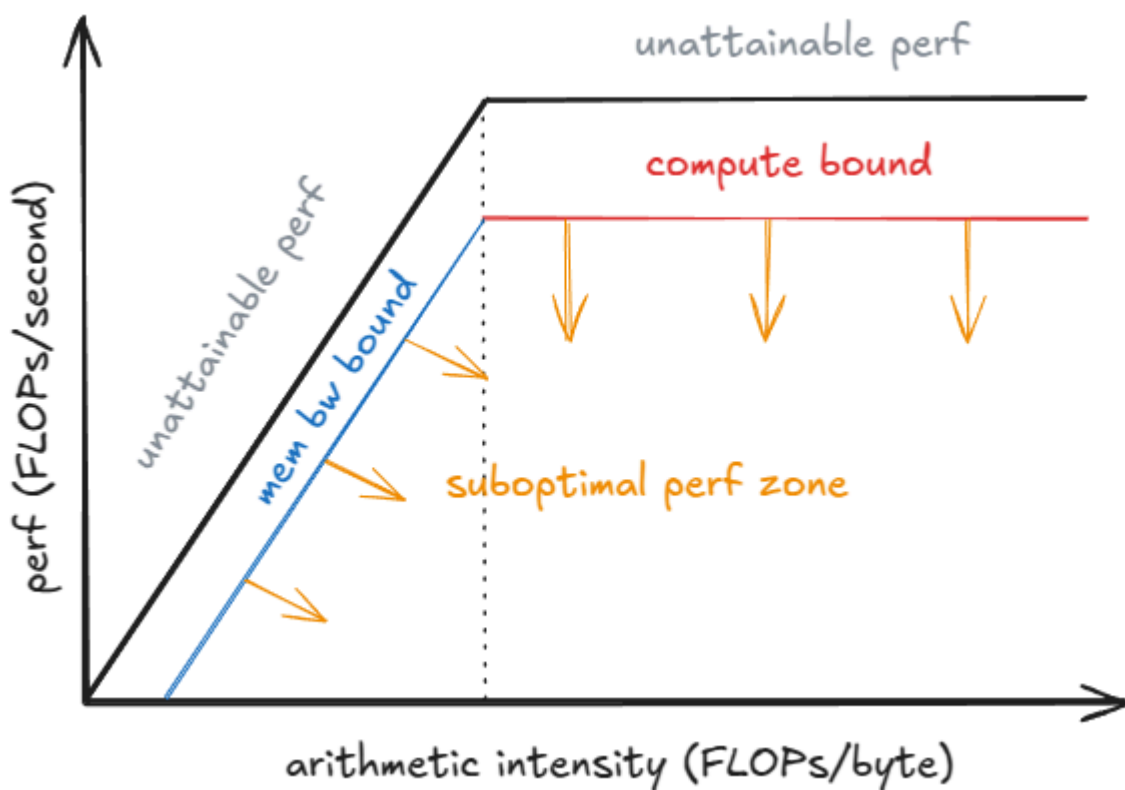
TTFT、ITL、E2E 延迟

这是一个简化的模型，解释了这两个指标的竞争性质。

假设：权重 I/O 而不是 KV 缓存 I/O 占主导地位；也就是说，我们正在处理短序列。

在查看批量大小如何影响单个解码步骤时，权衡变得清晰起来。至于 1，ITL 下降：每一步的工作更少，并且代币不会与其他代币“竞争”。至于无大，ITL 上升是因为我们每步执行更多的 FLOP，但吞吐量会提高（直到我们达到峰值性能），因为权重 I/O 在更多令牌上摊销。 $B \downarrow, B \uparrow$

屋顶线模型有助于理解这一点：在饱和批次以下，步进时间由 HBM 带宽主导（逐层将权重流式传输到片上存储器中），因此步进延迟几乎是平坦的——计算 1 个与 10 个令牌可能需要类似的时间。超过，内核变得受计算约束，步进时间大致随着 B_{sat} ；每个额外的代币都会添加到 ITL。



车顶线性能模型



注意：

为了进行更严格的处理，我们必须考虑内核自动调整：随着增长，运行时可能会切换到该形状的更高效的内核，从而改变实现的性能。步骤延迟是 $t = \frac{FLOPs_{step}}{P_{kernel}}$ ，其中 P_{kernel} 是步骤中的功。您可以看到，随着命中，每步的计算量增加将直接导致延迟增加。

如何在 vLLM 中进行基准测试

vLLM 提供了一个 CLI 来包装 `vllm / benchmarks / {server, latency, throughput}.py`。 `vllm bench {serve,latency,throughput}`

以下是脚本的作用：

- **延迟** – 使用短输入（默认 32 个令牌）并对 128 个输出令牌进行小批量采样（默认为 8 个）。它运行多次迭代并报告批处理的 e2e 延迟。
- **吞吐量** – 一次提交一组固定的提示（默认：1000 个 ShareGPT 示例）（又名模式），并在整个运行过程中报告每秒的输入/输出/总令牌和请求数。 `QPS=Inf`

- **serve** – 启动 vLLM 服务器，并通过从 Poisson（或更一般地说是 Gamma）分布中采样请求到达间隔时间来模拟现实世界的工作负载。它在一个时间窗口内发送请求，测量我们讨论过的所有指标，并且可以选择强制执行服务器端最大并发性（通过信号量，例如将服务器限制为 64 个并发请求）。

下面是如何运行延迟脚本的示例：

```
vllm bench latency
--model <model-name>
--input-tokens 32
--output-tokens 128
--batch-size 8
```

CI 中使用的基准配置位于 `.buildkite/nightly-benchmarks/tests`

还有一个自动调整脚本，它驱动服务基准测试以查找满足目标 SLO 的参数设置（例如，“最大化吞吐量，同时保持 p99 e2e < 500 毫秒”），返回建议的配置。

结语

我们从基本的引擎核心（）开始，添加了推测解码和前缀缓存等高级功能，扩展到（with），最后横向扩展，将所有内容包装在异步引擎和分布式服务堆栈中，最后以如何衡量系统性能结束。 `UniprocExecutor MultiProcExecutor TP/PP > 1`

vLLM 还包括我跳过的专门处理。例如：

- 多样化的硬件后端：TPU、AWS Neuron (Trainium/Inferentia) 等。
- 架构/技术：，，编码器-解码器（例如，Whisper）、池化/嵌入模型、，，、无注意力变体、滑动窗口注意力、多模态 LM 和状态空间模型（例如 Mamba/Mamba-2、Jamba） `MLA MoE EPLB m-RoPE LoRA ALiBi`
- TP/PP/SP
- 混合 KV 缓存逻辑（Jenga）、更复杂的采样方法（如波束采样）等
- 实验性的：异步调度

好消息是，其中大多数都与上述主流正交——您几乎可以将它们视为“插件”（当然，在实践中有一些耦合）。

我喜欢理解系统。话虽如此，决议在这个高度肯定受到了影响。在接下来的文章中，我将放大特定的子系统并深入了解细节。

 联系我们：

小红书

小红书号：95420586205

如果您发现帖子中有任何错误，请私信我 - 请随时在 [X](#) 或 [LinkedIn](#) 上或通过[匿名反馈](#)给我留言。

确认

非常感谢 [Hyperstack](#) 在过去一年中为我的实验提供了 H100!

感谢 [Nick Hill](#) (RedHat 核心 vLLM 贡献者)、[Mark Saroofim](#) (PyTorch)、[Kyle Krannen](#) (NVIDIA、Dynamo) 和 [Ashish Vaswani](#) 阅读了本博文的预发布版本并提供了反馈!

引用

1. vLLM <https://github.com/vllm-project/vllm>
2. “注意力就是你所需要的”，<https://arxiv.org/abs/1706.03762>
3. “使用 PagedAttention 服务的大型语言模型服务的高效内存管理”，<https://arxiv.org/abs/2309.06180>
4. “DeepSeek-V2: 一种强大、经济、高效的混合专家语言模型”，<https://arxiv.org/abs/2405.04434>
5. “Jenga: 为具有异构性的 LLM 提供服务的有效内存管理”，<https://arxiv.org/abs/2503.18292>
6. “Orca: 基于 Transformer 的生成模型的分布式服务系统”，<https://www.usenix.org/conference/osdi22/presentation/yu>
7. “XGrammar: 面向大型语言模型的灵活高效的结构化生成引擎”，<https://arxiv.org/abs/2411.15100>
8. “利用推测采样加速大型语言模型解码”，<https://arxiv.org/abs/2302.01318>
9. “EAGLE: 推测抽样需要重新思考特征不确定性”，<https://arxiv.org/abs/2401.15077>
10. “美杜莎: 具有多个解码头的简单 LLM 推理加速框架”，<https://arxiv.org/abs/2401.10774>
11. LMCache, <https://github.com/LMCache/LMCache>